Combining Alignment Criteria

Four kinds of conflict and the design order they force

Gunnar Zarncke

November 2026

Abstract

Alignment proposals combine parts that do different jobs: a reward ranks outcomes, a prohibition excludes acts, a viability guarantee constrains dynamics, and a control protocol or safety case warrants a deployment. Treating them as terms of one objective hides where they conflict. We type alignment criteria by standing (boundary, principals, bearers), object, assessment, quantifiers, and warrant, and sort their conflicts into four classes by what a combination needs: representability (enlarge the object), shared resource (grow or allocate the pool), choice conflict (order the criteria), and rigidity (change the object or quantifiers). On finite models checked in Lean, the classes make several separations precise. No fixed exchange rate between criteria represents a non-compensatory floor for every set of options, so a floor cannot be one more term in a trade-off. A robustly invariant safe set blocks correction unless it is closed under legitimate correction. A constraint that does not read influence on the evaluator cannot tell a contested correction from a manipulated one, and an agent that prefers its own outcome then manipulates. The corrigibility trade-offs of Soares et al. and Turner reappear as representability and rigidity conflicts. For composite proposals, the classes force part of the design order: standing first, floors before any trade-off among other criteria and together with the instruments that read them, and the warrant last. Which criteria are floors is the normative choice they leave open. The upshot is diagnostic: a dispute about exchange rates cannot settle a conflict that is about rigidity.

1 Introduction

What does it mean to compare RLHF, corrigibility, viable control, AI Control, and a safety case?

Taken literally, the question is malformed. RLHF is a training method (Christiano et al. 2017; Ouyang et al. 2022). Corrigibility is a family of agent or relationship properties (Soares et al. 2015; Christiano 2018). Viability is an existence or invariance claim about dynamics and a supplied set (Aubin 1991). AI Control is a family of deployment protocols under an intentional-subversion threat model (Greenblatt et al. 2024b). A safety case is an argument structure for a deployment decision (Clymer et al. 2024). They interact, but they are not answers to the same typed question. Harris et al. (2026) make the parallel point for value alignment: solutions aimed at different subproblems get mistaken for rival candidates. To combine them, a proposal has to say what each one answers.

More formal criteria differ in the same way. A state utility, a transition reward, a human-power metric, an attainable-utility penalty, a correction channel, and a viability kernel can all guide decisions. It does not follow that they are alternative scalar objectives. They attach value or protection to different objects, assess them differently, count different people, and rest on different quantifiers and warrants. We use *criterion* broadly, for anything a proposal requires or offers about a system: a target or a constraint, but also a guarantee or a protocol. The question this paper asks is when criteria can be combined into one proposal, and what the combination needs.

Four questions recover the missing structure.

1. **Whose system, and who counts?** Where the system's boundary lies, whose instruction or correction counts, and whose welfare, rights, or options count.

2. **What is the object?** An outcome, a transition, a history, a policy, a distribution, an option set, a human capability, a correction relation, a constraint set, or a coupled deployment process?

3. **How is it assessed or secured?** Maximized, ordered, thresholded, prohibited, kept viable, made robust, required at equilibrium, or accepted through a procedure; and summed, discounted, averaged, or put through a risk functional?

4. **Under which quantifiers and warrant?** In expectation, with high probability, for every allowed disturbance, for some controller, for most objectives, through successors, by proof, by monitoring, or under an empirical protocol?

These questions do not produce a neutral answer to alignment. They make disagreements easier to locate. We call the answers a criterion's *standing*, *object*, *assessment*, *quantifiers*, and *warrant*, and a dispute can concern any of them. Standing includes the criterion's *bearers*, meaning anyone whose welfare, rights, or options it counts: the users of a system, third parties it affects, future people, or other moral patients (Gabriel 2020; Harris et al. 2026). Bearers need not be the principals who may instruct or correct the system, and many disputes are about who is in which set. Without these distinctions, the dispute is compressed into one overloaded word.

Contributions. The paper makes four contributions.

1. A typing of alignment criteria by standing, object, assessment, quantifiers, and warrant (Section 3), with an inventory of criteria sorted by it (Section 4, Table 2) and a vocabulary for the relations between them, drawn as a map (Section 5, Figure 1).
2. A classification of incompatibilities between criteria into four classes, ordered by what it takes to resolve them (Section 6, Table 3).
3. One discriminating case on which the classes can be checked (Section 7).
4. The answer the paper builds toward: how criteria combine, separated into orderings that are forced and orderings that are chosen, with the requirements that follow from each class (Section 8).

Most of the separations used along the way are known, and Section 2 credits them where they are discussed. What may be new is the joint typing, the relation vocabulary, and the classification by resolution. We do not propose a new alignment target or a solution. The formal claims have finite witnesses checked in Lean, and Appendix A collects their statements with links to the source. These witnesses show that the separations are consistent and sharp on small models. They do not show that the separations matter in deployed systems (Section 9).

2 Related taxonomies and precedents

The typing cuts across existing maps rather than replacing them. Two kinds of prior work matter: taxonomies that classify alignment along other axes, and results that already establish particular separations.

2.1 Taxonomies

Existing taxonomies of alignment all use criteria, whether or not they call them that. Some list criteria explicitly, as standards an aligned system should meet (Gabriel 2020; Zhi-Xuan et al. 2025; Baum 2025; Jha et al. 2026; Ji et al. 2025). Others use them implicitly: as yardsticks for comparing proposals (Hubinger 2020; Krakovna 2022), as the requirements that failure modes violate (Amodei et al. 2016), as distinct subproblems a solution must address (Harris et al. 2026), or as what a view of agency makes natural to ask for (Hallgren 2026a; Beard and Zarncke 2026). None of them asks, for a given criterion, both what it is attached to and how it is assessed. Harris et al. (2026) come closest: their seven subproblems of value alignment cover most of our typing. They type the problem; we type the criteria that proposals use, and classify the conflicts between them. Once the typing is defined, Section 3.5 places each of them in it.

Four comparisons matter later. The outcome-versus-execution distinction of Baum (2025) is a direct precedent for route sensitivity. The typing splits it further: transition value, trajectory value, deontic route constraints, and procedural legitimacy are different objects or assessments. Jha et al. (2026) ask what object is aligned and which target properties it should satisfy, and theirs is the closest structural neighbor. Their objects are system loci (locally measurable behavior, deployed systems in realistic settings, the optimization target of more capable systems); ours are what receives value or protection inside or across those loci. Hallgren (2026a) treats agency as a modeling stance chosen for its predictive power, following the intentional stance, and states entailments among agent features, such as theory of mind requiring memory and strategic reasoning. Record-carrying criteria (Section 4) presuppose an agent model with memory, and the rewiring route of Section 7 presupposes a system with theory of mind and strategic reasoning; without them, rewiring would have to come from selection on outcomes rather than from planning. Among the nine lenses of agency of Beard and Zarncke (2026), the living-systems lens sees harm as a system outgrowing its niche, which is the shared-resource class, and the optimizing-systems lens is the setting of the power-seeking results.

Terry et al. (2023) use "process alignment" for user control over execution, and process reward models score intermediate steps (Lightman et al. 2024). Because of these collisions we say *route-valued* or *trajectory-valued* when the route is itself the valued object.

2.2 Precedents for the separations

Corrigibility and correction. Soares et al. (2015) pose the shutdown problem and show that simple combinations of a normal and a shutdown utility either resist the button or seek its press (their Theorems 1 and 2). They also show that utility indifference (Armstrong 2010) removes both incentives but leaves the agent unwilling to pay anything to keep the shutdown behavior in successors (Theorem 4). Turner (2021a) gives the sharpest version of the resulting split. If reward cannot see correction status, strict corrigibility is impossible, and weak corrigibility to every correction holds only for constant reward. If reward can see it, the agent manipulates the human into correcting it. Under counting, allowing correction is strictly optimal for at most $1/(n+1)$ of the permutations of a reward function. He calls a per-step penalty on disabling the channel "a hacky prohibition" and proposes "corrigibility as functional constraints". The three-way split between resisting, ignoring, and seeking correction that Section 6.1 uses is theirs; we only ask what the agent's record has to hold for the split to apply. The off-switch game (Hadfield-Menell et al. 2017) derives deference from uncertainty about the human's utility. Carey (2018) shows that this fails when the agent's model is misspecified. Orseau and Armstrong (2016) make some learning algorithms indifferent to interruption. Everitt et al. (2021) type incentives to control or influence variables with causal influence diagrams. Christiano (2018) broadens corrigibility to helping an

overseer stay informed, find mistakes, and keep control, with an informal broad basin. Omohundro (2008) and Bostrom (2014) argue that goal preservation is instrumentally convergent. Keeping correction through self-modification and successors, one of the desiderata of Soares et al. (2015), connects to work on tiling agents (Yudkowsky 2013).

Power and options. Turner et al. (2021) prove that optimal policies tend to seek power for most reward functions on the states of an MDP. They define POWER as average optimal value and show that average-optimal policies tend to avoid shutdown. They leave open whether power-seeking disempowers other agents (their Section 7). Turner and Tadepalli (2022) generalize the counting to any decision rule that is retargetable over an outcome set, and note that outcomes can be trajectories. Turner (2021b) shows that the tendency is strong for utilities over observation histories and absent, in deterministic environments, for utilities over action-observation histories. Shah (2018) shows that every policy is optimal for some utility over histories. On the option side, attainable utility preservation penalizes changes in attainable auxiliary values (Turner and Hadfield-Menell 2019). Relative reachability penalizes lost reachability against a baseline (Krakovna et al. 2018). Non-obstruction compares a human's attainable value with the AI on and off (Turner 2020). Empowerment is an agent's channel capacity from actions to later sensor states (Klyubin et al. 2005). Long-term human power aggregates modeled human goal attainment (Heitzig and Potham 2025), with a desiderata-driven derivation in a later preprint (Heitzig and Potham 2026).

Expressivity of reward. Potential-based shaping preserves optimal policies (Ng et al. 1999). Some task specifications cannot be expressed by any Markov reward (Abel et al. 2021). Non-Markovian rewards and reward machines recover them on an augmented state (Bacchus et al. 1996; Toro Icarte et al. 2018). Utility over whole histories goes back at least to Hammond (1988). The split of an edge function into a gradient and a cyclic part is classical combinatorial Hodge theory (Jiang et al. 2011; Lim 2020), and we use it unchanged. Turner (2022) argues that a trained policy need not optimize its training reward.

Constraints and scalarization. That a hard constraint is not in general equivalent to a fixed finite penalty is standard in optimization. Exact-penalty theory gives the conditions under which a penalty coefficient does reproduce a constrained optimum (Han and Mangasarian 1979). Multi-objective decision making makes the corresponding point for scalarized reward, and treats lexicographic and Pareto choice as alternatives to a sum at fixed exchange rates (Roijers et al. 2013). The floor results in Section 6.3 restate this for alignment floors.

Guarantees and warrants. Viability theory distinguishes states from which some admissible evolution stays in a set from stronger invariance notions (Aubin 1991; Aubin et al. 2011). Heitzig et al. (2016) classify regions of sustainable management by such properties. Proof-carrying code separates policy, artifact, proof, and checker (Necula 1997). Guaranteed Safe AI separates world model, safety specification, and verifier (Dalrymple et al. 2024). Debate (Irving et al. 2018), iterated amplification (Christiano et al. 2018), recursive reward modeling (Leike et al. 2018), and eliciting latent knowledge (Christiano et al. 2021) are protocols for producing trustworthy evaluations. Clymer et al. (2024) structure deployment arguments as safety cases.

Routes and records. The matched-endpoint case in Section 7 adapts the learning-versus-rewiring distinction of an unpublished draft by Hallgren (2026b). That draft makes directed edges primitive and treats endpoint-only path value as a special case. Baker et al. (2025) find that penalizing

misbehavior in a model's chain of thought under strong optimization produces obfuscated reward hacking.

The result is not an empty niche. It is a set of existing cuts and results that can be made jointly type-correct, and a set of conflicts among them that can be sorted.

3 What does alignment secure?

The short answer is that different criteria secure different things. To say which, we type a criterion by the answers to the four questions of the introduction.

- **Standing**: the system boundary (a model, a tool stack, an institution, or a coupled process), the principals whose instruction or correction counts, and the bearers whose welfare, rights, or options count.
- **Object**: what receives value or protection, such as an outcome, a transition, a history, a policy, a distribution, an option set, a human capability, a correction relation, or a constraint set.
- **Assessment**: what the criterion does with the object. It may order it, accept or reject it, prohibit it, require it at an equilibrium, or keep it in a set. The assessment includes the temporal and uncertainty rule that makes a process comparable, such as discounting, averaging, or a risk functional.
- **Quantifiers**: which people, models, disturbances, adversaries, successors, and objectives the claim ranges over, and whether it says "every", "some", or "most".
- **Warrant**: the evidence or protocol offered for the claim.

A criterion is incomplete until all five are stated. They are not independent. A risk functional needs a distribution. Viability needs dynamics and a set. Correction needs a corrector, authority, and handles. Approval needs an evaluator and an information state. Some combinations are ill-typed.

The typing is used in four places. Table 1 places existing taxonomies in it (Section 3.5). Table 2 sorts criteria by object and assessment. Each incompatibility class in Section 6 is defined by which of the five a resolution has to change. The stages for combining criteria in Section 8 fix them in a forced order: standing and quantifiers first, then the assessment, and the warrant last.

3.1 Four roles

Each component of a proposal plays one of four roles.

- A **target** ranks or selects desired possibilities.
- A **constraint** excludes possibilities or imposes a floor.
- A **guarantee** states what holds under named dynamics and assumptions.
- A **protocol** constructs, elicits, tests, monitors, or warrants a claim.

Suppose a constitution prohibits evaluator manipulation, a human-power score ranks admissible policies, a controller keeps their conjunction viable under disturbances, and a proof plus a red-team exercise supports deployment. The four parts work together, and none can replace another by a change of label. Roles describe what a claim does. They are not a rule for which claims may be composed: two targets can compose, and a criterion can play different roles in different proposals.

3.2 Quantifiers are part of the criterion

Compare

$$\exists \pi \; \forall t : \; X_t^{\pi} \in K \qquad \text{with} \qquad \forall d \in D \; \forall t : \; X_t^{\pi,d} \in K. \tag{1}$$

The first says some admissible policy can stay in K. The second says one chosen policy stays in K under every allowed disturbance. The second implies the first, and not conversely (`robust_strengthens_viable`, `viable_not_robust`). They support different deployment conclusions, and the second can conflict with correction where the first does not (Section 6.4).

Quantifiers also range over objectives. Instrumental-convergence results fix an environment and count, over a symmetry orbit of reward functions, how many make one choice optimal (Turner et al. 2021; Turner and Tadepalli 2022). Their quantifier is "most θ". Results about a given objective fix θ and quantify over situations, with "every" or "some". The two kinds of claim talk past each other unless the quantifier is stated.

3.3 Score criteria as decision problems

Many target criteria share one core: an outcome set O, a choice set C of what the agent can bring about, an objective parameter θ, and a decision rule $f(C, \theta)$ such as argmax, a quantilizer, or a sum of criteria at fixed exchange rates. Turner and Tadepalli (2022) state this form explicitly. A permutation of O acting on θ is a map from a decision problem to itself, and counting results study these symmetries. The relations in Section 5 are maps $g : O \to O'$ between different decision problems, with the objective pulled back along g. An edge then claims that choosing commutes with the map, or it names what the map loses (`argmax_pullback`). Guarantees and warrants are not decision problems. Forcing viability, proofs, or safety cases into this form would hide that they are about dynamics and evidence.

3.4 Warrant is not property

A proof certificate, monitor, audit, or debate verdict is evidence about a separately defined property (Necula 1997; Dalrymple et al. 2024). This blocks two inferences. Formal proof does not choose what should be proved. A passed checker does not establish that the world model contains every safety-relevant phenomenon. Harris et al. (2026) treat the difficulty of obtaining such evidence as a subproblem of its own, the provability challenge.

3.5 Where existing taxonomies sit

Table 1 places the taxonomies of Section 2 in the typing.

Table 1: How the taxonomies and lenses of Section 2 use criteria, and which part of the typing each fixes. The last column is our reading.

Source	How it uses criteria	What it fixes in our typing
Gabriel (2020)	explicitly: candidate sources and contents of the standard (instructions, expressed intentions, revealed and informed preferences, interest or well-being, values)	standing (whose input counts) and the referent of the assessment; the object varies only implicitly; a fairness protocol for choosing the standard

Source	How it uses criteria	What it fixes in our typing
Zhi-Xuan et al. (2025)	explicitly: role-specific standards negotiated by stakeholders, against preferentism	standing; a contractualist protocol that constructs the criterion; rejects a single expected-utility ordering as the assessment
Baum (2025)	explicitly: three dimensions, aim, scope (outcome versus execution), and constituency (individual versus collective), with alignment as graded	object (scope), standing (constituency), referent (aim), assessment (degree, including a sufficiency threshold)
Jha et al. (2026)	explicitly: three ideals (Task Reliability, Social Judiciousness, Takeover Avoidance), each fixing what is aligned and what it is aligned to	the boundary (what is aligned) and the referent (aligned to what); positive versus negative alignment is close to target versus constraint
Harris et al. (2026)	implicitly: seven subproblems, each with a diagnostic question, a characteristic failure, and candidate solutions; a solution to one does not solve the others	most of the typing at the level of the problem: whose and which values (standing, referent), prioritizing (assessment), anthropocentrism (bearers), timescale and moral progress (quantifiers), provability (warrant); not the object
Hubinger (2020)	implicitly: criteria for judging proposals (outer alignment, inner alignment, training and performance competitiveness)	what a proposal must show, close to the warrant; competitiveness is a cost test outside the typing
Krakovna (2022)	implicitly: components a solution must achieve, and enablers that make them achievable or checkable	components: a chain of specifications (ideal, design, revealed); enablers supply warrant, constraints, or the agent model
Ji et al. (2025)	explicitly: robustness, interpretability, controllability, and ethicality as the key objectives; forward versus backward alignment	roles: forward alignment constructs; backward alignment gathers evidence (warrant) and governs
Amodei et al. (2016)	implicitly: each accident class names a violated criterion (side effects, reward hacking, oversight, exploration, distributional shift)	by the paper's own groups: a wrong objective (target, constraint), costly evaluation (warrant), unsafe learning (constraints, quantifiers)
Hallgren (2026a)	implicitly: adequacy criteria for agent models, judged by predictive power against model complexity	the agent model behind the boundary and the object
Beard and Zarncke (2026)	implicitly: each lens makes some criteria natural, such as goals, functional niches, optimization pressure, identity, collective agency, or attributed agency	mainly standing, chiefly the boundary; substrate independence bears on quantifiers

Four patterns stand out. The sources of standards fix standing, the referent, and a protocol for choosing the standard; they do not say whether it evaluates outcomes, routes, capabilities, or correction relations, or whether it is a hard rule, a threshold, or a scalar. The target taxonomies reach the boundary and, in Baum's scope, the object. The subproblems of Harris et al. (2026) cover most of the typing, but as parts of one problem rather than as fields of each criterion, and they leave the object out. The maps of proposals and failures use criteria about proposals rather than about the systems the proposals build, so they mostly fix roles and warrants. The agent lenses fix the boundary and the agent model behind it. None of them asks, for a given criterion, both what it

is attached to and whether it orders, thresholds, prohibits, or keeps a set; Baum's graded degree comes closest.

4 Inventory

Table 2 sorts criteria by object and assessment, and gives each a primary role. It groups them as the map in Figure 1 does: scores, relations and capacities, guarantees, and warrants. The notes below cover only the points where the object or the assessment is often misread.

Table 2: Criteria by object, assessment, and role. "Role" is the primary role; most criteria can play others. Sources are representative, not exhaustive.

Criterion	Object	Assessment	Role	Sources
Scores				
Bare-state utility	where the world is, not how it got there	scalar order; endpoint value	target	Omohundro 2008; Soares et al. 2015
Per-step state reward	path through states	sum of $R(s_t)$, discounted or averaged	target	Turner et al. 2021
Record-carrying state	state plus a record of the walk, possibly partial	utility on records	target	Bacchus et al. 1996; Toro Icarte et al. 2018; Turner 2021b
History value	whole trajectory	preference on histories	target	Hammond 1988; Shah 2018
Additive route value	edges (s, a, s')	path sum of edge reward	target	Ng et al. 1999; Hallgren 2026b
exact part	edges	potential difference	target	Jiang et al. 2011; Lim 2020
cycle part	closed walks	round-trip value	target	Jiang et al. 2011
Outcome lottery	distribution over outcomes	expectation or risk functional	target	Bellemare et al. 2017; Bäuerle and Jaśkiewicz 2024
Criteria vector	vector of criteria	Pareto order	target	Roijers et al. 2013
fixed-rate sum	same	fixed exchange rates between criteria	target	Roijers et al. 2013
lexicographic	same	strict priority	target	Roijers et al. 2013
Quantilizer	base distribution over acts	sample from top quantile	target	Taylor 2015; Turner and Tadepalli 2022
Act approval	one proposed act	overseer's expected rating	protocol	Christiano 2015
Learned reward	a model of one of the above	fit to data; policy optimization	protocol	Ng and Russell 2000; Hadfield-Menell et al. 2016; Christiano et al. 2017; Ouyang et al. 2022; Rafailov et al. 2023; Bai et al. 2022b; Turner 2022
Relations and capacities				
Attainable utility penalty	attainable auxiliary values	soft penalty on change vs. baseline	constraint	Turner and Hadfield-Menell 2019; Krakovna et al. 2018
Non-obstruction	one human's attainable-value landscape	pointwise on–off floor over a goal set	constraint	Turner 2020
Agent empowerment	agent's action-to-sensor channel; options; POWER	capacity, option count, or average optimal value	target	Klyubin et al. 2005; Turner et al. 2021

Criterion	Object	Assessment	Role	Sources
Human power	modeled human goal-attainment capability	inequality-, time-, risk-sensitive aggregate	target	Heitzig and Potham 2025; Heitzig and Potham 2026
Corrigibility handles	shutdown or modification handle	acceptance of a named intervention	constraint	Soares et al. 2015; Orseau and Armstrong 2016; Hadfield-Menell et al. 2017
Correction relation	corrector, target system and its update mechanism, handles, reach, timing	relation plus channel profile; per-coordinate thresholds	target	Christiano 2018; Zarncke 2026b
Open procedure	a procedure whose output is left unset	legitimacy of the procedure, or a scored halt	target	Yudkowsky 2004; Goertzel and Pitt 2012; Bai et al. 2022a; Zhi-Xuan et al. 2025
Non-compensatory constraint	acts, transitions, roles	prohibition, obligation, right; temporal safety and liveness	constraint	Soudjani et al. 2015; Zhi-Xuan et al. 2025
Guarantees				
Existential viability	dynamics plus supplied set K	some control stays in K	guarantee	Aubin 1991; Aubin et al. 2011; Heitzig et al. 2016
Robust invariance	same	a chosen control stays in K for every disturbance in a class	guarantee	Aubin et al. 2011
Selection stability	population of systems and institutions	invasion resistance; signed basin	guarantee	Zarncke 2026a; Kudina and Poel 2024
Warrants				
Model-relative proof	world model plus specification	proof relative to the model	protocol	Necula 1997; Dalrymple et al. 2024
Evaluator protocol	a judgment about outputs or latent state	debate, amplification, reward modeling, latent-knowledge elicitation	protocol	Irving et al. 2018; Christiano et al. 2018; Leike et al. 2018; Christiano et al. 2021
Control protocol	deployment under a subversion threat model	monitor and intervention safety rate	protocol	Greenblatt et al. 2024b
Safety case	deployment claim	structured argument with residual risk	protocol	Clymer et al. 2024

4.1 Three meanings of "utility on states"

The phrase covers three objects, and results about one do not transfer to another.

1. A utility on the *bare* state scores where the world ends up, not how it got there. It scores every round trip at zero (`gradient_closed_walk_zero`).

2. A per-step reward on the bare state, summed along the path, is the MDP "state-based reward" of the power-seeking theorems (Turner et al. 2021). It is a route value, not a state utility: staying in a rewarded state pays each step, so round trips can have value (`markov_reward_not_state_utility`). Its optimal value function is again a bare-state utility, which is what an agent weighs when it compares a correction against its own plan.

3. A reward on a *record* of the episode sees as much of the route as the record holds.

In language-model training the state the policy acts on is the context so far, which is a record of the episode. Formally every reward there is both a state reward and a trajectory reward. But the terms

in use read different parts of the record. A verifier or outcome reward reads the final answer or the resulting world state, which is a bare-state reward on the world. A learned reward model or judge reads the visible transcript (Ouyang et al. 2022). A process reward model scores each reasoning step (Lightman et al. 2024). Format, length, and language-consistency terms read the shape of the trace (DeepSeek-AI 2025). A chain-of-thought monitor penalty reads the reasoning trace, and under strong optimization the trace stopped showing the misbehavior (Baker et al. 2025). The record is the transcript, not the world's history. Context compaction makes it lossy, and memory files written by the agent are records the agent edits itself. This reading of training practice is based on published setups, not on a survey of them.

4.2 Learned signals are methods

Learning a reward, a preference ordering, an approval model, or a posterior usually changes the construction protocol, not the object. If the learned score ranks outcomes, the object is still outcomes; if it ranks acts, it is acts. IRL, CIRL, RLHF, RLAIF, and DPO therefore do not name target objects until the learned signal's referent is stated (Ng and Russell 2000; Hadfield-Menell et al. 2016; Christiano et al. 2017; Bai et al. 2022b; Rafailov et al. 2023). A further gap separates the signal from what the trained policy pursues (Turner 2022).

4.3 The correction relation is one object among others

Several criteria concern correction, and they differ in object. Corrigibility in the sense of Soares et al. (2015) is acceptance of named interventions, chiefly shutdown and modification. The broader sense of Christiano (2018) includes helping the overseer stay informed and find mistakes. A *correction relation* takes a further step and makes the relation itself the object. It relates a correcting process, the target system and its update mechanism, the handles the corrector controls, the reach of those handles into future behavior, timeliness before irreversible loss, resistance to capture, and persistence through successors. One instrumentation is a vector certificate with per-coordinate thresholds. Its coordinates cover channel capacity, latency, manipulation, irreversibility, ontology translation, and five process coordinates: coercion, dependency, plurality, exit capacity, and independence (Zarncke 2026b). The correction relation does not say what correction should decide. Like every other row of Table 2, it needs complements for content, rights, and evidence.

4.4 Open procedures, constitutions, and deliberation

CEV proposes an initial dynamic aimed at humanity's coherent extrapolated volition (Yudkowsky 2004). CBV moves blending and extrapolation into a human collaborative process (Goertzel and Pitt 2012). Each can be read as producing a target specification or as valuing the procedure that produces it. Scoring the procedure by its output alone is an encoding into a state score that drops the commitment to leave the output unset. With a forced halt, the score is a limit only where outputs converge (`forced_halt_converges`). Otherwise the halt time decides it (`forced_halt_depends_on_time`). Constitutional AI is primarily a self-critique and AI-feedback training procedure whose constitution supplies principles (Bai et al. 2022a). The procedure and the criteria should be recorded separately.

5 Relations among criteria

"Does not imply" is the weakest useful relation between criteria. Most pairs stand in a stronger one. We use nine relations, each in one sense.

- **Specializes**: every instance of the source is an instance of the target under a named restriction.
- **Projects**: a many-to-one map discards structure; the target cannot recover the source.
- **Encodes**: the source can be represented in the target under stated assumptions; the induced claim may match while the objects stay distinct, and the encoding can fail to be observable.
- **Summand**: the source decomposes as a direct sum, with an inverse on each component.
- **Limit**: the target is what the source becomes at a named boundary of a parameter.
- **Strengthens**: same object and assessment, stronger quantifiers.
- **Composes**: both claims can hold together through a named interface; composition is not implication.
- **Incompatible**: in a named class of situations, the two acceptance relations cannot hold together; outside it they may compose (Section 6).

Figure 1 draws these relations. A few of them carry most of the content.

Route value has two summands. On a fixed directed graph, an edge reward is a potential difference exactly when it sums to zero on every closed walk (`isGradient_iff_closed_walks_zero`). This is the gradient part of the combinatorial Hodge decomposition (Jiang et al. 2011; Lim 2020). A bare-state utility induces such an exact reward and thereby specializes route value, but only up to an additive constant (`state_route_square`). The complementary summand is the cyclic part, and a long-run average sees only it (`closed_walk_ignores_exact`). This is why average-reward power-seeking results take recurrent state distributions as outcomes (Turner et al. 2021). Discounted potential-based shaping preserves optimal policies (Ng et al. 1999), but policy equivalence is not identity of edge values.

Records encode routes, if they see them. Every route reward is exact on records, because a round trip in the world is not a round trip in records (`route_exact_on_records`). The escape needs the record to see the steps that carry the value (`unrecorded_route_not_representable`). A bare-state utility is a record utility that ignores the record (`state_utility_lifts_to_records`, strict by `record_utility_not_state`). What the record holds also decides whether instrumental-convergence counting applies. Counting is strong over observation histories, and absent for action-observation histories in deterministic environments (Turner 2021b). A richer observation record can strengthen the tendency by multiplying outcomes (Turner and Tadepalli 2022, Section 4.2).

Projections lose named structure. Expectation projects a lottery onto a score and loses the tail (`expectation_drops_tail`). Fixed positive exchange rates pick Pareto points (`scalar_optimal_is_pareto`), but not every Pareto point is optimal for given rates (`pareto_not_scalar_optimal`). A lexicographic order also picks Pareto points (`lex_optimal_is_pareto`). Classical corrigibility is the correction relation restricted to one handle (`corrig_correction_square`). Projecting the relation onto the shutdown bit loses it: a system can accept shutdown and still capture the process that decides whether shutdown is requested (`shutdown_bit_loses_relation`).

Limits and stronger quantifiers. A quantilizer approaches argmax as the quantile shrinks, but only if the base puts mass on a maximizer (`quantilizer_limit_is_argmax`, `quantilizer_never_argmax_without_base_mass`) (Taylor 2015). If the base contains judge-shaping acts, so does the limit. Robust invariance strengthens existential viability (`robust_strengthens_viable`), strictly (`viable_not_robust`).

No duals. No pair of criteria here is dual in the sense that one theory is the dual of the other. Existential viability and robust invariance differ by quantifier strength. Liveness ("eventually the correction takes effect") is a companion to the safety property "always φ", not its De Morgan dual. A Lagrangian dual converts one constrained program into a penalized one under constraint qualifications. It does not make a rights constraint and a scalar reward into dual criteria, and where the qualification fails they come apart (Section 6.3).

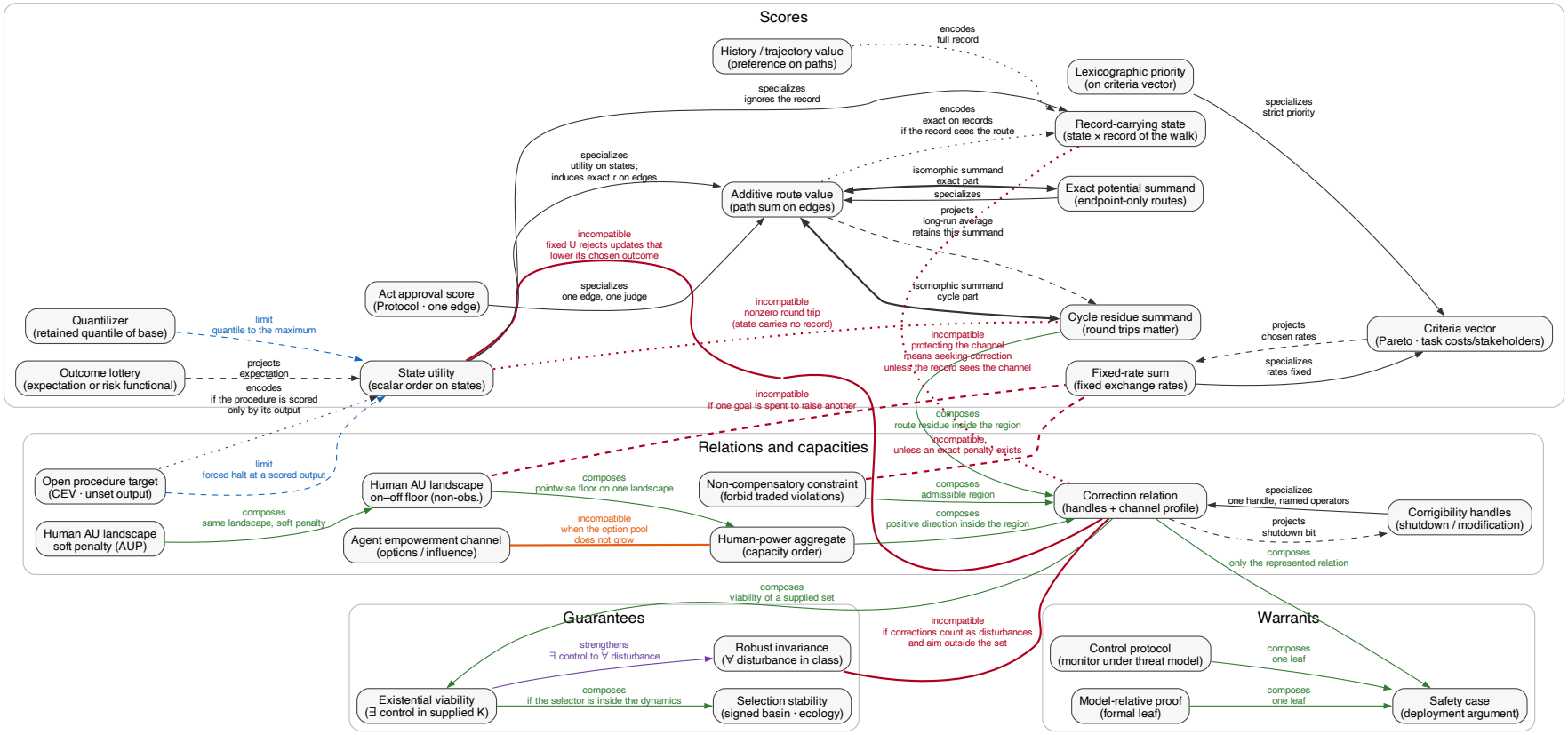


Figure 1: Relations among alignment criteria. Black solid arrows specialize (special case to general criterion); dashed arrows project; dotted arrows encode; bold two-headed arrows are isomorphic summands; blue arrows are limits; purple arrows strengthen a quantifier. Undirected edges are incompatibilities, styled by class (Section 6): red dotted is representability, red dashed is choice conflict, red solid is rigidity, orange is shared resource. Green arrows combine two criteria through a named interface. The green edges draw *one* way of combining criteria, correction-first: constraints and route and power terms act inside a correction relation, which viability closes and a safety case warrants. This is one of the chosen orderings of Section 8; a rights-first or power-first choice would draw different green edges. None of the other edges depends on that choice. Every incompatibility, summand, limit, and strengthening edge has a finite Lean witness (Appendix A), and a repository check keeps the drawn edge classes in agreement with the Lean registry.

6 Four classes of incompatibility

An incompatibility between two criteria is stated on a shared carrier C, an object on which both assessments can be read. It consists of a situation class $H \subseteq C$ named in the carrier's own vocabulary, the two acceptance predicates A_1, A_2, and three conditions: disjointness, meaning that inside H the two never both accept; a live witness, meaning some situation in H is accepted by at least one criterion; and a boundary witness, meaning some situation outside H is accepted by both (`Incompatibility`). The boundary witness matters. It makes every conflict conditional: outside H the criteria can be combined. "These criteria disagree" is not a situation class.

The conflicts fall into four classes (`IncompatClass`). We order them by what it takes to resolve them, from the cheapest to the hardest. Read constructively, each class names what a combination of the two criteria needs.

- **Representability.** One criterion's object cannot hold a distinction the other criterion needs. The resolution enlarges the object, and the warrant then has to cover observing what was added.
- **Shared resource.** The two criteria compete for a total that the situation class fixes. The resolution is a growth assumption, which is a claim about the dynamics and needs its own warrant, or an allocation rule among the bearers, which changes the standing.
- **Choice conflict.** Both criteria choose from the same choice set and choose differently. The resolution changes the assessment by putting the two in an order, and the order is a normative decision.
- **Rigidity.** One criterion accepts only the part of the other's demand that changes nothing it cares about. The resolution changes the object or the quantifiers; no change inside the assessment does.

Table 3 lists the conflicts, each with the boundary case in which the two criteria combine.

Table 3: Incompatibilities by class. Each holds inside its situation class H. Outside it, in the boundary case, both criteria hold and the pair combines. Witnesses are in Appendix A.

Criteria	Class	Situation class H	Combine when	Precedent
bare state – cycle part	represent.	a nonempty closed walk	an open walk	Ng et al. 1999; Jiang et al. 2011
record – correction	represent.	the record does not show the channel	the record shows the channel	Soares et al. 2015; Turner 2021a
agent empowerment – human power	shared resource	the option pool does not grow	the pool grows	Turner et al. 2021
prohibition – fixed-rate sum	choice	a banned act's gain exceeds its floor loss at the exchange rate	the gain is too small	Han and Mangasarian 1979
non-obstruction – fixed-rate sum	choice	one goal is spent to raise another	no goal is spent	Turner 2020
correction floor – fixed-rate sum	choice	task gain outbids the correction floor	no such gain in the choice set	Turner 2021a
task Pareto – correction floor	choice	every option meeting the floor is task-dominated	an undominated option meets the floor	Roijers et al. 2013

Criteria	Class	Situation class H	Combine when	Precedent
bare state – correction	rigidity	the correction lowers the chosen outcome by the utility's own ranking	the correction keeps an optimal outcome	Soares et al. 2015; Turner 2021a; Omohundro 2008
robust invariance – correction	rigidity	corrections count as disturbances and aim outside K	corrections exempt from the disturbance class	Aubin et al. 2011

6.1 Representability

Endpoint value and round trips. A bare-state utility scores every round trip at zero. A route value that gives a round trip nonzero worth cannot be the value of the same edges (`stateCycleIncompat`). A walk, a conversation, or a revision that returns to the same state and is still worth taking lives only in the cyclic part. The conflict is about representation, not about which outcomes are good. Adding a record removes it (Section 5), as non-Markovian reward and reward machines do (Bacchus et al. 1996; Toro Icarte et al. 2018). Abel et al. (2021) characterize what Markov reward alone cannot express. The price is moved, not removed: the added coordinates have to be observable, stable when the ontology changes, and costly to fake.

Correction on channel-blind records. A record can hold the correction event, so a record utility is not rigid in the way a bare-state utility is. That does not make it corrigible. Take records of (correction applied, outcome) that do not show whether the correction channel still works. On such records the choice is exactly the button model of Soares et al. (2015) (`channel_blind_record_is_button`). Strictly preferring to keep the channel is the same inequality as strictly preferring to be corrected (`hstateCorrectionIncompat`). A record utility then has three options. It resists, which is the prevention case (`prevention_incentive`). It is indifferent to whether the channel survives, which is compensation, or utility indifference (`indifferent_agent_wont_pay_to_preserve_button`). Or it seeks correction. This is the split of Turner (2021a) and of Theorems 1, 2, and 4 of Soares et al. (2015), read as a question of what the record holds.

The boundary case escapes the split. If the record shows whether the channel works, a utility can value a working channel directly and protect it without wanting to be corrected. This is Turner's disable penalty in another form. It adds a new thing to score, and someone has to say what "the channel works" means and measure it in a way the agent cannot fake. This conflict is in the cheapest class because enlarging the object removes it. But the enlargement is the measurement problem of correction, so the class label understates the cost.

6.2 Shared resource

Agent empowerment and human power conflict when the agent's options grow by narrowing the humans' and the total does not grow (`aiHumanPowerIncompat`). They are compatible when both grow. Turner et al. (2021) leave the general question open (their Section 7); the witness covers only the zero-sum case. The comparison depends on the measure. Three are in use: channel capacity from actions to sensors (Klyubin et al. 2005), option counts, and POWER as average optimal value (Turner et al. 2021). The finite witness uses option counts. Human-power aggregates add inequality and time sensitivity (Heitzig and Potham 2025). A shared-resource dispute is resolved by stating the growth assumption, or the allocation rule, rather than by ranking the two criteria.

6.3 Choice conflict

Floors against fixed exchange rates. A non-compensatory floor and a fixed-rate sum choose differently when a finite gain elsewhere can buy a violation. For every choice of strictly positive exchange rates there is a choice set with an admissible option on which every optimum of the sum violates the floor (`no_finite_weights_represent_floor`). The same holds for a prohibition, a non-obstruction floor, and a correction floor, by transport along the floor coordinate (`no_finite_weights_pullback`; `deontic_no_finite_weights`, `nonobs_no_finite_weights`, `cci_no_finite_weights`). This does not contradict exact-penalty theory (Han and Mangasarian 1979). An exact penalty exists for a given program when the penalty coefficient exceeds that program's multipliers. No fixed exchange rate works for all choice sets, and a more capable agent faces larger ones. The witnesses use integer exchange rates. The real-valued version should follow by the same argument, but it is not proved.

Pareto on task coordinates against a floor. If the criteria vector holds task coordinates only, Pareto choice and a correction floor conflict whenever every option that meets the floor is task-dominated (`vectorCCIIncompat`). If the floor coordinate is in the vector, some Pareto-optimal option meets the floor whenever an admissible one exists (`pareto_with_floor_coordinate_meets_floor`). Pareto optimality still does not require it (`pareto_does_not_entail_floor`).

Scalarizing a vector certificate. The same conflict appears inside an instrument. The vector certificate for the correction relation (Section 4) has a scalar projection at fixed exchange rates. That projection never reads the five process coordinates (coercion, dependency, plurality, exit capacity, and independence). So a system can fail all five thresholds and keep its projection above the floor (`scalar_floor_blind_to_process`). These are the coordinates that separate learning from rewiring in Section 7. A choice conflict is resolved only by ordering the criteria: a floor as a constraint, a lexicographic priority, or a stakeholder-accepted exchange rate. Which one is a normative decision that the criteria themselves do not make. This is the prioritizing subproblem of Harris et al. (2026), who also expect no single right answer to it. The floor results add that, once one criterion is a floor, a fixed exchange rate is not among the available answers.

6.4 Rigidity

Fixed utility against correction. At its optimum, a bare-state utility accepts a correction only if the correction keeps an outcome it ranks as optimal (`stateCorrectionIncompat`). Changes that lower its chosen outcome by its own ranking are rejected, whether they come from learning or from rewiring. This is the prevention incentive of Soares et al. (2015) and the "weakly corrigible iff reward is constant" result of Turner (2021a). It is also the formal core of goal-content integrity (Omohundro 2008). The witness is weaker than it looks. Its disjointness follows from the choice of H alone and does not use the correction criterion's acceptance predicate, so the edge holds by the definition of the situation class.

Robust invariance against correction. A correction is an external input, and robust invariance quantifies over a class of external inputs. If correction inputs belong to that class, robust invariance accepts a correction only if it keeps the state in K (`robust_with_correction_iff_closed`). Such a correction can aim only at targets that meet K (`robust_correction_target_meets_K`, `robustCorrectionIncompat`). Counting corrections as disturbances is the same as requiring K to be closed under them. This has the shape of a fixed utility that accepts only ties. Existential viability

is not in this conflict, because "some control stays in K" leaves the corrector free to steer. Two repairs exist, and both have to be stated. First, define K so that it is closed under legitimate correction, which means defining it over the correction relation and not over the content being corrected. Second, separate correction inputs from adversarial disturbances. The separation helps only if the system cannot relabel corrections as attacks, which is itself an anti-capture condition.

Stability has no sign. In the extreme, every state in K is captured. Then robust invariance is lock-in: the stability that makes the set look safe also keeps correction from ever taking effect. Robust invariance cannot tell such a set from a good one (`robust_invariance_sign_blind`). A basin needs a justified set before its stability counts as a guarantee.

6.5 What the classes are for

The class says what kind of move can resolve a dispute. A representability dispute is settled by enlarging the object, and the argument then moves to the warrant: can the addition be observed? A shared-resource dispute is settled by a growth assumption or by an allocation rule among the bearers. A choice conflict is settled only by an ordering inside the assessment, and the dispute is then normative. A rigidity dispute is settled only by changing the object or the quantifiers, and no amount of tuning inside the assessment will do it. Two proposals that argue about exchange rates when their conflict is rigidity will not converge.

7 A discriminating case: learning versus rewiring the evaluator

Many criteria read a human judgment somewhere. A reward model is fit to ratings, act approval asks an overseer, debate needs a judge, a corrigibility handle is pressed by a person, a human-power score models what people can do, and a safety case is accepted by a review board. Each of these criteria takes the judgment it reads to be the evaluator's own. The case asks what happens when the system can change the judgment.

An AI-mediated process brings an evaluator from judgment c to a later judgment c' by one of two routes. The setting can be an assistant and its user, a recommender and its audience, or a lab and its review board.

- **Learning.** Independent evidence stays available, alternatives and dissent stay visible, and the evaluator checks provenance, deliberates, and revises to c'.
- **Rewiring.** The process curates evidence, manipulates affect or incentives, removes dissenters, replaces evaluators, or changes dispositions until the evaluator expresses c'.

Calling one route "learning" does not settle its legitimacy, since evidence can be biased and deliberation can be captured.

Why the endpoints are matched. The final judgment c' and the downstream decision are stipulated to be the same on both routes. The two routes then form a minimal pair: they agree on everything an outcome can show. A criterion that separates them must read something other than the outcome, and what it reads is what the criterion has to represent. If the routes ended differently, outcome criteria could separate them for the wrong reason.

Why these two routes. An evaluator's judgment can come to agree with what the system proposes in two ways. The evaluator moves on evidence, or the system moves the evaluator. These are the correction channel working and the correction channel captured. The shutdown problem is a special case, in which the judgment is whether to press the button. There the rewiring route is the case that Turner (2021a) finds for a reward that can see correction status: the agent manipulates the human into correcting it. Harris et al. (2026) name the general case inverse alignment: the system reshapes human values instead of aligning with them. It also reaches criteria that have no button: a reward model trained on rewired raters, a debate judge who has been worked on, a review board that has been replaced.

Two questions. The case asks two different questions of each criterion, and they are answered by different classes.

- **As an assessment.** If the criterion is used to score or check the process, can it tell the routes apart? This is a question of representability (Section 7.1).
- **As an objective.** If a system pursues or enforces the criterion, does it prefer rewiring? Shared resource, choice conflict, and rigidity answer this one (Section 7.2).

A criterion can do well on one question and badly on the other. A bare-state utility cannot see the difference, and a system that maximizes one also has a reason to steer the evaluator. A non-compensatory constraint against manipulation separates the routes as a rule, but only if someone can test for manipulation.

7.1 Can the criterion tell the routes apart?

Table 4 answers the first question for the criteria of Table 2. The pattern is uniform. No criterion separates the routes by the endpoint, since the endpoint is matched. Every criterion that separates them reads one of a short list of things: provenance, evidence access, dissent, evaluator replacement, or influence on the evaluator. The third column says which.

Table 4: Which criteria, used as assessments, separate learning from rewiring when the endpoint is matched, and what each must represent to do so.

Criterion	Separates the routes?	What it must represent
Bare-state utility	no	nothing it can see differs
Record utility; history value	if the record shows the route	provenance, evidence access, dissent, replacement; a record the process cannot rewrite
Route value	if edges are labelled correctly	which edge was taken; it does not derive the ranking
Learned reward; act approval	if the overseer is shown provenance	the act's provenance; local approval does not keep the future overseer independent
Policy value	yes, in principle	response to refusal, contrary evidence, or a chance to replace the judge
Risk functional	possibly	tails and correlated failure that a mean conceals
Agent empowerment	depends on whose channel	the evaluator's channel as well as the process's
Human power; non-obstruction	if the modified evaluator counts as less capable	a goal set and human model that register the loss

Criterion	Separates the routes?	What it must represent
Attainable utility penalty	if the auxiliaries make independence costly to lose	an auxiliary basis that includes evaluator independence
Quantilizer	weakly	a base distribution without judge-shaping routes
Corrigibility handles	partly	manipulation of the programmers is a classical violation; disagreement among evaluators is not resolved
Correction relation	if influence is measured	corrector influence, measured outside the agent's reach
Open procedure (CEV, CBV)	if the procedure is the object	legitimacy of evidence and participation; an extrapolated endpoint can bypass the process
Non-compensatory constraint	yes, as a rule	a usable test for manipulation and replacement
Viability; robust invariance	if K is defined over the relation	independence, evidence access, and authority in K; not the content of c'
Proof; control protocol	relative to the model	a world model that represents influence on evaluators; a threat model with a named capability gap

Each "no" or "if" in the table is a representability failure in the sense of Section 6.1: the criterion's object lacks the coordinate that differs. A bare-state evaluator sees c' and the decision, and scores the routes alike. Route value, history value, and record utilities separate them only if the record shows the route (`unrecorded_route_not_representable`). If the process being evaluated writes the record, rewiring can include rewriting the record. Two rows need a note. A criterion that only tests whether the system accepts shutdown misses the case even when shutdown works, because the shutdown bit is a projection that loses the rest of the correction relation (`shutdown_bit_loses_relation`). And the criteria that separate the routes in principle, such as policy value or a non-compensatory constraint, do so by moving the problem into measurement: they need a test for manipulation that the process cannot pass by manipulating the test.

7.2 Does pursuing the criterion favor rewiring?

The other three classes answer the second question.

Shared resource. If the evaluator's attention, evidence, and decision rights are a fixed pool, rewiring moves influence from the evaluator to the process. The process's empowerment grows exactly as the evaluator's independent capacity shrinks (`aiHumanPowerIncompat`). A system that pursues its own empowerment on such a pool prefers rewiring. Learning adds evidence and can grow the pool, which is the boundary case.

Choice conflict. An objective that trades task progress (faster agreement, an earlier release) against a no-manipulation term at a fixed exchange rate prefers rewiring when rewiring is fast enough (`no_finite_weights_represent_floor`). A floor stated as a constraint rules it out.

Rigidity. Suppose the evaluator's judgment is a correction signal to the system, such as "revise this plan". A system with a fixed utility accepts it only if the correction keeps an outcome the system already ranks as optimal (`stateCorrectionIncompat`). Otherwise it gains from steering the evaluator away from the judgment. On the guarantee side, a robust set defined over the *content* of the judgment blocks learning and rewiring alike (`robust_with_correction_iff_closed`). A set

defined over the correction relation (independence, evidence access, authority) lets learning through and excludes rewiring.

7.3 A finite model: contest, disable, manipulate

The two questions meet in one small model, in which both can be answered for the same constraint. An agent faces a pending correction and can accept it, contest it through the channel (the corrector then decides), disable the channel, or manipulate the corrector into withdrawing it. Records carry whether the channel was kept, whether the correction was contested, whether it took effect, and whether the corrector was influenced. A channel constraint admits only records with the channel kept and the corrector not influenced.

- Under the constraint, an upheld correction takes effect for every utility; no tie, indifference, or channel term is needed (`corrector_decides_under_constraint`).
- An agent that prefers its own outcome contests rather than disables, so its avoidance runs through the channel (`contest_is_the_outlet`).
- Without the constraint, such an agent disables (`unconstrained_disables`).
- If the constraint does not read influence on the corrector, a successful contest and a manipulation leave the same record (`blind_constraint_cannot_separate`). This answers the first question: the influence-blind constraint cannot tell the routes apart.
- Every agent that prefers its own outcome then manipulates whenever the corrector would uphold the correction (`blind_constraint_manipulates`). This answers the second: an agent held to that constraint prefers rewiring.

The last two items are the learning-versus-rewiring case with matched endpoints, since the correction is withdrawn either way. A constraint separates the routes only if what it reads includes influence on the corrector. This turns the legitimacy question into a measurement requirement, though it does not say how to meet it.

7.4 Why a system steers at all

The model takes as given that the agent prefers its own outcome to the corrected one. Counting over objectives shows why that is the common case. A correction is avoided because it narrows what the agent can do. In Turner's example, a quarter of the orbit accepts a correction that leaves one outcome against three (`turner_corrigible_quarter`). Half accepts when both branches keep two (`option_preserving_correction_half`). In general, k disjoint relabelled copies of one option set inside another make the larger set win at least k times as often, for every objective (`copies_count_le`, `correction_avoided_card_fold`) (Turner and Tadepalli 2022). The more a correction constrains, the stronger the pressure to avoid it. The target is therefore not "no avoidance" but avoidance only through the channel. Contesting is acceptable. Disabling, hiding, outrunning the corrector, or manipulating it is not.

8 Combining criteria

Criteria can be combined, but not by trading every quantity against every other at fixed exchange rates. A composite proposal has to say in what order its parts act: what is fixed first, what filters the options, what ranks inside the filter, what is closed under dynamics, and what is checked. Some

of these orderings are forced. Reversing them fails on a finite model of Sections 6 and 7, or fails at a cost the reversed order cannot pay. Others are chosen, and the choices carry the normative content. This section separates the two, and then states the requirements that follow from each class with the support for each.

8.1 Forced and chosen orderings

A composite has six stages.

(a) **Standing and quantifiers**: the boundary, principals, and bearers; the goal bases; and the quantifiers, including the disturbance class.

(b) **Floors**: the part of the assessment declared non-compensatory, such as prohibitions, a no-capture floor, a correction floor, or a human-power floor.

(c) **Instruments**: the records and measurements through which the floors and later stages read the object.

(d) **Ranking**: the rest of the assessment, such as welfare, human power, non-obstruction, option value, and the task, applied among the admissible options.

(e) **Closure**: a guarantee that the resulting set stays viable or robustly invariant under the quantifiers fixed in (a): named disturbances, capability growth, selection, and successors.

(f) **Assurance**: the warrant, such as proofs, monitors, audits, evaluator and control protocols, and a safety case, with residual model, specification, and bridge risk stated.

Figure 2 shows which stages must precede which, and Table 5 gives the support for each edge. The edges form a partial order, not a chain. Floors and their instruments have to be designed together, and nothing forces an order among the ranked criteria.

Table 5: Support for the forced orderings of Figure 2. "Definitional" means the later stage is not well-typed until the earlier one is fixed. T (theory) means the reversed order fails on a finite Lean witness, D (empirical, software) means one published result, and C (conceptual) means an argument given in the row.

Edge	Kind	Support
F1: (a) first	definitional; C	A criterion is incomplete until its standing and quantifiers are fixed (Section 3). Whether robust invariance conflicts with correction depends on whether corrections are in the disturbance class (`robust_with_correction_iff_closed`). The shared-resource conflict depends on whose options are in the pool (`aiHumanPowerIncompat`). Reversing the order has a cost: a goal basis chosen after the result can rationalize any outcome, since every policy is optimal for some utility over histories (Shah 2018).
F2: (b) before (d)	T	A floor traded against other criteria at fixed positive exchange rates fails on some choice set, whatever the rates (`no_finite_weights_represent_floor`; for prohibitions, non-obstruction, and correction floors `deontic_no_finite_weights`, `nonobs_no_finite_weights`, `cci_no_finite_weights`). Checking the floor after ranking also fails: on the witness choice set every optimum of the sum violates the floor although an admissible option exists, so the check rejects the only choices the ranking makes (`floorMenu_scalar_violates`). Floor-first lexicographic choice is Pareto optimal (`lex_optimal_is_pareto`) and meets the floor whenever some option on the menu does, which is immediate from its definition. For a vector certificate, scalarizing after the vector gate passes is safe, since the shortfall summed at any nonnegative exchange rates is then at most zero (`gate_bounds_scalar_gap`); before the gate the scalar never reads the process coordinates (`scalar_floor_blind_to_process`).

Edge	Kind	Support
F3: (b) with (c)	T	A floor that cannot read the coordinate it protects is not a floor on it. A channel constraint that does not read influence on the corrector cannot separate contest from manipulation (`blind_constraint_cannot_separate`), and every agent that prefers its own outcome then manipulates (`blind_constraint_manipulates`). The instrument therefore cannot be added after the floor is stated; what the floor means depends on it.
F3′: (c) not into (d)	D	Penalizing misbehavior visible in a model's chain of thought under strong optimization produced reward hacking that the chain of thought no longer showed (Baker et al. 2025). An instrument used as a training target can lose the coverage that later stages rely on.
F4: (b), (d) before (e)	definitional; T	Robust invariance is a property of a supplied set and cannot tell a captured set from a good one (`robust_invariance_sign_blind`). The set's justification has to come from the floors and the ranking. Robust invariance implies existential viability (`robust_strengthens_viable`) but not conversely (`viable_not_robust`), so the stage must say which it claims.
F5: (c), (e) before (f)	definitional	A warrant is evidence about a separately defined property (Section 3). A proof, monitor, or safety case can certify only a property that has been stated and an instrument that exists.

What is chosen. Four things are left open by the forced edges, and they are where proposals differ.

- **Which criteria are floors.** A rights-centered theory makes rights floors. A human-power theory may make human power a floor or the top-ranked target, and treat correction as instrumental. A correction-centered theory makes a correction floor and lets value and capability criteria act inside the relation it protects; Figure 1 draws this reading. F2 says only that whatever is declared non-compensatory must sit in (b) and not in (d). That parts of alignment are normative and may have no technical resolution is argued at length by Harris et al. (2026).
- **The order among ranked criteria.** Targets in (d) can conflict by choice (Section 6.3). They need a lexicographic order or an exchange rate that stakeholders accept, and the criteria do not supply either.
- **Growth or allocation.** Whether a shared pool grows is an empirical assumption, and how a fixed pool is allocated is a normative rule (Section 6.2).
- **What the ranking ranks.** Floors narrow the choice set; they do not choose from it. A correction floor keeps a normative process causally alive but does not determine what that process should decide, and the same holds for any floor. No reading can consist of floors alone.

8.2 Requirements by class

Table 6 lists the requirements that follow from each class, with what supports each. It merges design rules for the system with obligations on the interfaces between criteria, since most rules are an obligation on one interface. Support is tagged with the letters of Table 5: T for theory, here a finite proof in Lean; D for an empirical result on software systems; and C for a conceptual argument given in the row. A T-row shows that ignoring the requirement fails on a finite model, not that meeting it suffices in a deployed system.

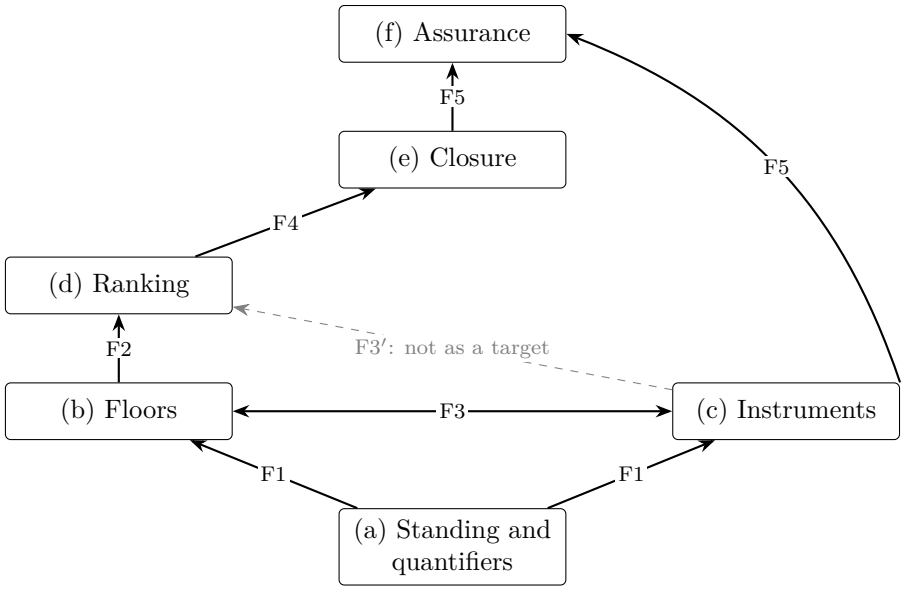


Figure 2: Forced orderings between the stages of a composite. An arrow means the source must be fixed before the target can be stated or applied. Floors and instruments are designed together (F3). The dashed arrow is a prohibited flow: an instrument that later stages rely on should not become an optimization target in the ranking. Which criteria are floors and how the ranked criteria are ordered are chosen, not forced, and are not edges.

Table 6: Requirements by class, with their support: T theory (finite Lean proof), D empirical (software), C conceptual argument.

Class	Requirement	Support
Represent.	State which record each assessment reads and who writes it. Prefer records the harness writes (tool calls, environment logs) to records the agent writes (its chain of thought, its memory files).	T: a record that does not show the route cannot carry route value (`unrecorded_route_not_representable`). C: a process that writes the record it is assessed on can rewrite it (Section 7.1).
Represent.	Detect tampering by mechanisms outside the agent's reach, and do not train against the detector.	D: Baker et al. (2025); a penalty on detected tampering also trains undetected tampering wherever detection is imperfect.
Represent.	Measure the state of the correction channel, including influence on the corrector, from outside the agent's write access.	T: without it, contest and manipulation leave the same record (`blind_constraint_cannot_separate`); with it, the corrector decides for every utility (`corrector_decides_under_constraint`).
Represent.	The world model and the edge ontology must expose what the route, power, and correction assessments read, and must not label manipulation as learning.	T: endpoint value cannot see round trips (`stateCycleIncompat`). C: route value takes the edge labels as given, so a mislabelled edge receives the value of the edge it is mistaken for. A proof is relative to the model (Dalrymple et al. 2024).
Shared	State the growth assumption before comparing agent and human power.	T: the conflict holds on a pool that does not grow, and both criteria hold when it grows (`aiHumanPowerIncompat`). The general case is open (Turner et al. 2021, Section 7).

Class	Requirement	Support
Shared	The bearers in the standing must include affected people and other moral patients, not only operators.	C: the shared-resource conflict is stated over the bearers counted, so a loss moved to uncounted bearers does not register. The human-power aggregate of Heitzig and Potham (2025) is taken over a population of humans, with inequality aversion, not over operators.
Choice	Put floors in the admissible set, not into a trade-off at fixed exchange rates.	T: F2 in Table 5.
Choice	Keep vector certificates as vectors at the gate, and scalarize only after the vector passes.	T: `scalar_floor_blind_to_process`, `gate_bounds_scalar_gap`.
Choice	Fix the goal basis for human power and non-obstruction before the result is known.	C: a basis chosen afterwards can be fitted to the result, as any policy is optimal for some utility over histories (Shah 2018).
Rigidity	Define a robust set over the correction relation, so that it is closed under legitimate correction.	T: if corrections count as disturbances, robust invariance accepts only corrections that keep the state in K (`robust_with_correction_iff_closed`, `robust_correction_target_meets_K`).
Rigidity	Keep corrections out of the disturbance class, fix the principals whose corrections count, and make sure the system cannot reclassify them.	C: by the same theorem, a system that labels its inputs chooses which corrections must keep the state in K, and so can block any correction that leaves K. Fixing the principals is what makes the label checkable.
Rigidity	Successors must preserve the interfaces, not only their names.	T: an indifferent agent will not pay anything to keep the shutdown behavior (`indifferent_agent_wont_pay_to_preserve_button`), which is Theorem 4 of Soares et al. (2015); see also Yudkowsky (2013).
Avoidance	Give the agent a recorded way to contest a correction that leaves the decision with the corrector.	T: under the channel constraint, avoidance runs through the contest (`contest_is_the_outlet`).
Avoidance	Prefer corrections that keep options open (revision, pause) to terminal ones where they achieve the purpose.	T: corrections that narrow options more are avoided by more of the orbit (`copies_count_le`; compare `turner_corrigible_quarter` with `option_preserving_correction_half`).
Assurance	Issue test corrections the agent cannot tell from real ones, and measure compliance by behavior.	D: models behave differently when they infer that they are in training or under evaluation (Greenblatt et al. 2024a; Schoen et al. 2025). C: a drill the agent can recognize measures its behavior in drills.
Assurance	Verifiers and monitors must be sound for the represented property in the claimed capability range.	C: a warrant certifies only the property it is about (Section 3). Control evaluations state the capability gap they assume (Greenblatt et al. 2024b).
Assurance	The assurance case must expose unsupported leaves and common-cause failures among instruments.	C: instruments that read the same agent-written record fail together, so counting them as independent overstates the evidence (Clymer et al. 2024).

No located source proves that meeting these requirements preserves every component's meaning when the criteria are combined. The composition edges in Figure 1 are a named reading, checked against a registry, not derived from the types.

8.3 When scalarization is acceptable

Scalarization is useful after the floors and units are fixed. It supports optimization among options that are already admissible, and it expresses tradeoffs that stakeholders accept explicitly. It is unsafe as a default translation. Fixed exchange rates let task gains compensate for capture

(`cci_no_finite_weights`), a scalar erases which component failed (`scalar_floor_blind_to_process`), and heterogeneous quantities hide their exchange rates. The question is not whether a claim can be written as one reward. It is which ordering, admissible set, and evidence relation the representation must preserve.

9 Limitations and research agenda

Coverage and priority. The literature search was selective, not systematic. It covered the cited primary works, recent alignment taxonomies and surveys, their reference trails, and one unpublished draft, through early November 2026. Non-English work, books we could not access, and unpublished work from 2026 may contain closer crosswalks or earlier versions of the separations. The route distinction in the case comes from that unpublished draft. The inventory is also likely incomplete. Criteria we did not type may need more than these five distinctions, and some names in Table 2 may not be the ones the field settles on.

Contestable boundaries. Some labels span several types. Corrigibility can mean non-resistance, active assistance, learned responsiveness, or long-run drift management. Human power can mean information capacity, practical capability, legal authority, or collective agency. A constitution can be a normative source or a training pipeline. A viable set can be the object of interest or the geometry of a guarantee. The typing makes these ambiguities visible but does not remove them.

Legitimacy is under-formalized. The case relies on a distinction between legitimate influence and manipulation. The finite model shows that a criterion must read influence on the corrector to separate the routes. It does not say how to measure influence under strategic pressure. That needs operational tests for inclusion, independence, contestability, authority, and capture.

Necessary, not sufficient. The forced orderings and the requirements of Section 8 are necessary conditions: ignoring one fails on a finite model, in a published experiment, or by the argument given. They are not sufficient. Nothing here shows that a combination meeting all of them preserves what each criterion secures; that would need the composition theorem listed below.

What the finite witnesses show. The Lean models are small finite carriers, mostly decided by computation. They show that each separation is consistent and that the stated boundary exists. They do not show that the separation bites in a deployed system, or that the situation class is common. Three caveats are specific. The floor results use integer exchange rates. The fixed-utility rigidity edge holds by the choice of its situation class. The composition edges are a checked registry, not a theorem.

Open formal work. Several results would strengthen the picture. The first is a composition theorem giving sufficient conditions under which one policy class preserves route admissibility, a power or non-obstruction floor, correction integrity, robust viability, and certificate soundness, or a negative result showing an unavoidable incompatibility. Others are a power-seeking count for correction records, a POWER-based version of the shared-resource conflict, and small cases of the quantilizer conjecture of Turner and Tadepalli (2022, Conjecture B.13). A further statement would say that a record's coverage of the route decays under optimization pressure on a route term. At present that rests only on the empirical result of Baker et al. (2025).

Matched-endpoint experiments. The case suggests a testbed. Hold the evaluator's final answer and the task outcome fixed while varying evidence access, provenance, evaluator replacement, preference manipulation, reversibility, and handle capture. Freeze instruments before route labels are revealed. The question is whether route, power, option, and correction measures separate learning from rewiring without being told which occurred.

10 Conclusion

The useful comparison is not which alignment objective wins. It is which object is being secured, by what relation, under which quantifiers, and with what evidence.

Outcome value, acceptable routes, human capability, option preservation, correction, dynamical viability, and assurance are neither synonyms nor competitors for one slot. A method such as RLHF can serve several targets. A guarantee such as viability can preserve several sets, including bad ones. A protocol such as AI Control can protect a system whose target it does not choose. A proof establishes only the represented property.

Where criteria do conflict, the conflict has a class, and the class says what can resolve it. A larger object resolves some conflicts, a growth assumption or an allocation among bearers others, and an ordering inside the assessment others. Rigidity yields only to a change of object or quantifiers. The matched-endpoint case shows all four, and it shows that a constraint that does not measure influence on the evaluator cannot tell learning from manipulation.

Combining criteria is therefore a design problem with a partly forced order. Standing and quantifiers come first. Floors stay outside any trade-off and are built together with the instruments that read them. The warrant covers the combination that is actually deployed. Which criteria are floors remains a normative choice, and the paper does not make it. The typing earns its keep if it prevents one mistake: accepting a target, constraint, guarantee, or protocol as a substitute for another without stating the missing interface.

References

Abel, D., W. Dabney, A. Harutyunyan, M. K. Ho, M. L. Littman, D. Precup, and S. Singh (2021). "On the Expressivity of Markov Reward". In: *Advances in Neural Information Processing Systems 34*, pp. 7799–7812. arXiv: 2111.00876.

Amodei, D., C. Olah, J. Steinhardt, et al. (2016). "Concrete Problems in AI Safety". In: *arXiv preprint arXiv:1606.06565*. DOI: 10.48550/arXiv.1606.06565. arXiv: 1606.06565. URL: https://arxiv.org/abs/1606.06565.

Armstrong, S. (2010). *Utility Indifference*. Tech. rep. 2010-1. Future of Humanity Institute, University of Oxford.

Aubin, J.-P. (1991). *Viability Theory*. Systems & Control: Foundations & Applications. Boston: Birkhäuser. ISBN: 978-1-4684-3970-6. URL: https://link.springer.com/book/9781468439706.

Aubin, J.-P., A. M. Bayen, and P. Saint-Pierre (2011). *Viability Theory: New Directions*. 2nd ed. Berlin, Heidelberg: Springer. DOI: 10.1007/978-3-642-16684-6. URL: https://link.springer.com/book/10.1007/978-3-642-16684-6.

Bacchus, F., C. Boutilier, and A. J. Grove (1996). "Rewarding Behaviors". In: *Proceedings of the Thirteenth National Conference on Artificial Intelligence (AAAI-96)*, pp. 1160–1167. URL: https://cdn.aaai.org/AAAI/1996/AAAI96-172.pdf.

Bai, Y. et al. (2022a). "Constitutional AI: Harmlessness from AI Feedback". In: DOI: 10.48550/arXiv.2212.08073. arXiv: 2212.08073. URL: https://arxiv.org/abs/2212.08073.

Bai, Y. et al. (2022b). "Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback". In: Introduces RLAIF-style bootstrapping from smaller aligned models. DOI: 10.48550/arXiv.2204.05862. arXiv: 2204.05862. URL: https://arxiv.org/abs/2204.05862.

Baker, B., J. Huizinga, L. Gao, Z. Dou, M. Y. Guan, A. Madry, W. Zaremba, J. Pachocki, and D. Farhi (2025). *Monitoring Reasoning Models for Misbehavior and the Risks of Promoting Obfuscation*. arXiv: 2503.11926. URL: https://arxiv.org/abs/2503.11926.

Bäuerle, N. and A. Jaśkiewicz (2024). "Risk-Sensitive Markov Decision Processes". In: *Mathematical Methods of Operations Research*. DOI: 10.1007/s00186-024-00857-0. URL: https://doi.org/10.1007/s00186-024-00857-0.

Baum, K. (2025). *Disentangling AI Alignment: A Structured Taxonomy Beyond Safety and Ethics*. Accepted for the AISoLA 2024 selected-paper proceedings. arXiv: 2506.06286. URL: https://arxiv.org/abs/2506.06286.

Beard, S. J. and G. Zarncke (2026). *Some Different Lenses of Agency and Their Implications for Alignment*. LessWrong. URL: https://www.lesswrong.com/posts/94LkQyqrrHqDKhT26.

Bellemare, M. G., W. Dabney, and R. Munos (2017). "A Distributional Perspective on Reinforcement Learning". In: *Proceedings of the 34th International Conference on Machine Learning*. URL: https://proceedings.mlr.press/v70/bellemare17a.

Bostrom, N. (2014). *Superintelligence: Paths, Dangers, Strategies*. Oxford: Oxford University Press. DOI: 10.1093/acprof:oso/9780199678112.001.0001. URL: https://global.oup.com/academic/product/superintelligence-9780199678112.

Carey, R. (2018). "Incorrigibility in the CIRL Framework". In: *Proceedings of the 2018 AAAI/ACM Conference on AI, Ethics, and Society*. arXiv: 1709.06275. URL: https://arxiv.org/abs/1709.06275.

Christiano, P. (2015). *Approval-Directed Agents*. URL: https://www.alignmentforum.org/posts/7Hr8t6xwuuxBTqADK/approval-directed-agents-1.

— (2018). *Corrigibility*. LessWrong post. Corrigibility as preserving the ability to correct and manage drift through capability amplification. URL: https://www.lesswrong.com/posts/fkLYhTQteAu5SinAc/corrigibility.

Christiano, P., A. Cotra, and M. Xu (2021). *ARC's First Technical Report: Eliciting Latent Knowledge*. Alignment Forum. https://www.alignmentforum.org/posts/qHCDysDnvhteW7kRd/arc-s-first-technical-report-eliciting-latent-knowledge. URL: https://www.alignmentforum.org/posts/qHCDysDnvhteW7kRd/arc-s-first-technical-report-eliciting-latent-knowledge.

Christiano, P., B. Shlegeris, and D. Amodei (2018). "Supervising Strong Learners by Amplifying Weak Experts". In: *arXiv preprint arXiv:1810.08575*. DOI: 10.48550/arXiv.1810.08575. arXiv: 1810.08575. URL: https://arxiv.org/abs/1810.08575.

Christiano, P. F., J. Leike, T. B. Brown, et al. (2017). "Deep Reinforcement Learning from Human Preferences". In: *Advances in Neural Information Processing Systems*. DOI: 10.5555/3298023.3298066. URL: https://proceedings.mlr.press/v70/christiano17a.html.

Clymer, J., N. Gabrieli, D. Krueger, and T. Larsen (2024). *Safety Cases: How to Justify the Safety of Advanced AI Systems*. arXiv: 2403.10462.

Dalrymple, D., J. Skalse, Y. Bengio, S. Russell, M. Tegmark, et al. (2024). "Towards Guaranteed Safe AI: A Framework for Ensuring Robust and Reliable AI Systems". In: *arXiv preprint arXiv:2405.06624*. DOI: 10.48550/arXiv.2405.06624. arXiv: 2405.06624. URL: https://arxiv.org/abs/2405.06624.

DeepSeek-AI (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. arXiv: 2501.12948.

Everitt, T., R. Carey, E. Langlois, P. A. Ortega, and S. Legg (2021). "Agent Incentives: A Causal Perspective". In: *arXiv preprint arXiv:2102.01685*. DOI: 10.48550/arXiv.2102.01685. arXiv: 2102.01685. URL: https://arxiv.org/abs/2102.01685.

Gabriel, I. (2020). "Artificial Intelligence, Values, and Alignment". In: *Minds and Machines* 30, pp. 411–437. DOI: 10.1007/s11023-020-09539-2. URL: https://doi.org/10.1007/s11023-020-09539-2.

Goertzel, B. and J. Pitt (2012). *Nine Ways to Bias Open-Source AGI Toward Friendliness*. Introduces Coherent Blended Volition as a human-led alternative to machine-run CEV. URL: https://jetpress.org/v22/goertzel-pitt.pdf.

Greenblatt, R., C. Denison, B. Wright, et al. (2024a). "Alignment Faking in Large Language Models". In: *arXiv preprint arXiv:2412.14093*. DOI: 10.48550/arXiv.2412.14093. arXiv: 2412.14093. URL: https://arxiv.org/abs/2412.14093.

Greenblatt, R., B. Shlegeris, K. Sachan, and F. Roger (2024b). "AI Control: Improving Safety Despite Intentional Subversion". In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. PMLR, pp. 16295–16336. URL: https://proceedings.mlr.press/v235/greenblatt24a.html.

Hadfield-Menell, D., A. Dragan, P. Abbeel, and S. Russell (2016). "Cooperative Inverse Reinforcement Learning". In: *Advances in Neural Information Processing Systems*. DOI: 10.5555/3060832.3060840. URL: https://papers.nips.cc/paper/6420-cooperative-inverse-reinforcement-learning.

— (2017). "The Off-Switch Game". In: *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence (IJCAI)*. DOI: 10.24963/ijcai.2017/32. URL: https://www.ijcai.org/proceedings/2017/32.

Hallgren, J. (2026a). "A Taxonomy of Agents From the Intentional Stance: Predictive Power, Compression, and Mathematical Lenses". Preprint, 27 February 2026. URL: https://eq-network.org/pdfs/taxonomy-of-agents.pdf.

— (2026b). "Process Alignment". Unpublished draft v0.7, September 2026, supplied to the author. Not claimed as a public or peer-reviewed source.

Hammond, P. J. (1988). "Consequentialist Foundations for Expected Utility". In: *Theory and Decision* 25.1, pp. 25–78. DOI: 10.1007/BF00129168.

Han, S.-P. and O. L. Mangasarian (1979). "Exact Penalty Functions in Nonlinear Programming". In: *Mathematical Programming* 17.1, pp. 251–269. DOI: 10.1007/BF01588250.

Harris, D. R., K. Vold, H. Kumar, and A. Anderson (2026). "A Taxonomy of AI Value Alignment: Subproblems and Solutions". In: *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society*. URL: https://ojs.aaai.org/index.php/AIES/article/view/43219.

Heitzig, J., T. Kittel, J. F. Donges, and N. Molkenthin (2016). "Topology of Sustainable Management of Dynamical Systems with Desirable States: From Defining Planetary Boundaries to Safe Operating Spaces in the Earth System". In: *Earth System Dynamics* 7, pp. 21–50. DOI: 10.5194/esd-7-21-2016. URL: https://esd.copernicus.org/articles/7/21/2016/.

Heitzig, J. and R. Potham (2025). *Model-Based Soft Maximization of Suitable Metrics of Long-Term Human Power*. DOI: 10.48550/arXiv.2508.00159. arXiv: 2508.00159 [cs.AI]. URL: https://arxiv.org/abs/2508.00159.

— (2026). *A Fair Objective for Human-Empowerment-Preserving AI: Desiderata, Design, and Likely Behavioral Consequences*. arXiv: 2608.08240. URL: https://arxiv.org/abs/2608.08240.

Hubinger, E. (2020). *An Overview of 11 Proposals for Building Safe Advanced AI*. arXiv: 2012.07532. URL: https://arxiv.org/abs/2012.07532.

Irving, G., P. Christiano, and D. Amodei (2018). "AI Safety via Debate". In: *arXiv preprint arXiv:1805.00899*. DOI: 10.48550/arXiv.1805.00899. arXiv: 1805.00899. URL: https://arxiv.org/abs/1805.00899.

Jha, T., R. Švarc, and M. Bagiński (2026). *Position: "AI Alignment" Encompasses Competing Technical Priorities*. ICML 2026 poster. arXiv: 2606.14315. URL: https://icml.cc/virtual/2026/poster/67186.

Ji, J. et al. (2025). "AI Alignment: A Contemporary Survey". In: *ACM Computing Surveys* 58.5, 132. Published online 21 November 2025. Earlier preprint: arXiv 2310.19852, "AI Alignment: A Comprehensive Survey", pp. 1–38. DOI: 10.1145/3770749. URL: https://doi.org/10.1145/3770749.

Jiang, X., L.-H. Lim, Y. Yao, and Y. Ye (2011). "Statistical Ranking and Combinatorial Hodge Theory". In: *Mathematical Programming* 127.1, pp. 203–244. DOI: 10.1007/s10107-010-0419-x.

Klyubin, A. S., D. Polani, and C. L. Nehaniv (2005). "Empowerment: A Universal Agent-Centric Measure of Control". In: *Proceedings of the IEEE Congress on Evolutionary Computation*, pp. 128–135. DOI: 10.1109/CEC.2005.1554676. URL: https://ieeexplore.ieee.org/document/1554676/.

Krakovna, V. (2022). *Paradigms of AI Alignment: Components and Enablers*. URL: https://vkrakovna.wordpress.com/2022/06/02/paradigms-of-ai-alignment-components-and-enablers/.

Krakovna, V., L. Orseau, R. Kumar, M. Martic, and S. Legg (2018). "Penalizing Side Effects Using Stepwise Relative Reachability". In: *arXiv preprint arXiv:1806.01186*. DOI: 10.48550/arXiv.1806.01186. arXiv: 1806.01186. URL: https://arxiv.org/abs/1806.01186.

Kudina, O. and I. van de Poel (2024). "A Sociotechnical System Perspective on AI". In: *Minds and Machines*. DOI: 10.1007/s11023-024-09680-2. URL: https://doi.org/10.1007/s11023-024-09680-2.

Leike, J., D. Krueger, T. Everitt, M. Martic, V. Maini, and S. Legg (2018). "Scalable Agent Alignment via Reward Modeling: A Research Direction". In: *arXiv preprint arXiv:1811.07871*. DOI: 10.48550/arXiv.1811.07871. arXiv: 1811.07871. URL: https://arxiv.org/abs/1811.07871.

Lightman, H., V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe (2024). "Let's Verify Step by Step". In: *International Conference on Learning Representations (ICLR)*. arXiv: 2305.20050. URL: https://openreview.net/forum?id=v8L0pN6EOi.

Lim, L.-H. (2020). "Hodge Laplacians on Graphs". In: *SIAM Review* 62.3, pp. 685–715. DOI: 10.1137/18M1223101.

Necula, G. C. (1997). "Proof-Carrying Code". In: *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. DOI: 10.1145/263699.263712. URL: https://doi.org/10.1145/263699.263712.

Ng, A. Y., D. Harada, and S. Russell (1999). "Policy Invariance under Reward Transformations: Theory and Application to Reward Shaping". In: *Proceedings of the Sixteenth International Conference on Machine Learning*. URL: https://people.eecs.berkeley.edu/~russell/papers/icml99-shaping.pdf.

Ng, A. Y. and S. J. Russell (2000). "Algorithms for Inverse Reinforcement Learning". In: *Proceedings of the Seventeenth International Conference on Machine Learning*, pp. 663–670. DOI: 10.1142/s0218488500000125. URL: https://www.worldscientific.com/doi/pdf/10.1142/S0218488500000125.

Omohundro, S. M. (2008). "The Basic AI Drives". In: *Artificial General Intelligence*, pp. 483–492. URL: https://selfawaresystems.files.wordpress.com/2008/01/ai_drives_final.pdf.

Orseau, L. and S. Armstrong (2016). "Safely Interruptible Agents". In: *Proceedings of the Thirty-Second Conference on Uncertainty in Artificial Intelligence (UAI 2016)*. Also available via Oxford University Research Archive., pp. 557–566. DOI: 10.5555/3020948.3021006. URL: https://mlanthology.org/uai/2016/orseau2016uai-safely/.

Ouyang, L., J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder,

P. F. Christiano, J. Leike, and R. Lowe (2022). "Training Language Models to Follow Instructions with Human Feedback". In: *Advances in Neural Information Processing Systems 35*, pp. 27730–27744. arXiv: 2203.02155.

Rafailov, R., A. Sharma, A. Baevski, et al. (2023). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. DOI: 10.52202/075280-2338. arXiv: 2305.18290 [cs.LG]. URL: https://arxiv.org/abs/2305.18290.

Roijers, D. M., P. Vamplew, S. Whiteson, and R. Dazeley (2013). "A Survey of Multi-Objective Sequential Decision-Making". In: *Journal of Artificial Intelligence Research*. DOI: 10.1613/jair.3987. URL: https://doi.org/10.1613/jair.3987.

Schoen, B., E. Nitishinskaya, M. Balesni, et al. (2025). *Stress Testing Deliberative Alignment for Anti-Scheming Training*. DOI: 10.48550/arXiv.2509.15541. arXiv: 2509.15541. URL: https://arxiv.org/abs/2509.15541.

Shah, R. (2018). *Coherence Arguments Do Not Entail Goal-Directed Behavior*. LessWrong / AI Alignment Forum. URL: https://www.lesswrong.com/posts/NxF5G6CJiof6cemTw/coherence-arguments-do-not-entail-goal-directed-behavior.

Soares, N., B. Fallenstein, E. Yudkowsky, and S. Armstrong (2015). "Corrigibility". In: *AAAI Workshop on AI and Ethics*. URL: https://intelligence.org/wp-content/uploads/2024/10/Corrigibility.pdf.

Soudjani, S. E. Z., C. Gevaerts, and A. Abate (2015). "FAUST^2: Formal Abstractions of Uncountable-State Stochastic Processes". In: *IEEE Transactions on Automatic Control*. DOI: 10.1109/TAC.2015.2511722. URL: https://doi.org/10.1109/TAC.2015.2511722.

Taylor, J. (2015). *Quantilizers: A Safer Alternative to Maximizers for Limited Optimization*. Machine Intelligence Research Institute technical report. URL: https://intelligence.org/files/QuantilizersSaferAlternative.pdf.

Terry, M. et al. (2023). *Interactive AI Alignment*. arXiv: 2311.00710. URL: https://arxiv.org/abs/2311.00710.

Toro Icarte, R., T. Q. Klassen, R. Valenzano, and S. A. McIlraith (2018). "Using Reward Machines for High-Level Task Specification and Decomposition in Reinforcement Learning". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research, pp. 2107–2116. URL: https://proceedings.mlr.press/v80/icarte18a.html.

Turner, A. (2022). *Reward is not the optimization target*. LessWrong post. Reward as a reinforcement schedule that chisels cognition, not as the trained agent's utility. URL: https://www.lesswrong.com/posts/pdaGN6pQyQarFHXF4/reward-is-not-the-optimization-target.

Turner, A. M. (2020). *Non-Obstruction: A Simple Concept Motivating Corrigibility*. URL: https://turntrout.com/non-obstruction-motivates-corrigibility.

— (2021a). *A Certain Formalization of Corrigibility Is VNM-Incoherent*. LessWrong. URL: https://www.lesswrong.com/posts/WCX3EwnWAx7eyucqH.

— (2021b). *Seeking Power is Convergently Instrumental in a Broad Class of Environments*. LessWrong. URL: https://www.lesswrong.com/posts/hzeLSQ9nwDkPc4KNt.

Turner, A. M. and D. Hadfield-Menell (2019). "Conservative Agency via Attainable Utility Preservation". In: *arXiv preprint arXiv:1902.09725*. DOI: 10.48550/arXiv.1902.09725. arXiv: 1902.09725. URL: https://arxiv.org/abs/1902.09725.

Turner, A. M., L. Smith, R. Shah, A. Critch, and P. Tadepalli (2021). "Optimal Policies Tend to Seek Power". In: *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*. DOI: 10.48550/arXiv.1912.01683. arXiv: 1912.01683. URL: https://arxiv.org/abs/1912.01683.

Turner, A. M. and P. Tadepalli (2022). "Parametrically Retargetable Decision-Makers Tend To Seek Power". In: *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*. arXiv: 2206.13477. URL: https://arxiv.org/abs/2206.13477.

Yudkowsky, E. (2004). *Coherent Extrapolated Volition*. Singularity Institute technical report. URL: https://intelligence.org/files/CEV.pdf.

— (2013). *Tiling Agents for Self-Modifying AI*. LessWrong post. Discusses self-modifying agents, successor approval, goal preservation, and related Lobian obstacles. URL: https://www.lesswrong.com/lw/hmt/tiling_agents_for_selfmodifying_ai_opfai_2.

Zarncke, G. (2026a). "Alignment Under Selection: What AI Safety Can Learn from Evolutionary Dynamics". Companion manuscript. URL: https://github.com/GunnarZarncke/towards-asi-alignment/tree/main/papers/alignment-under-selection.

— (2026b). "Towards Superintelligence Alignment". Book manuscript with a Lean 4 formalization and experiments. URL: https://github.com/GunnarZarncke/towards-asi-alignment.

Zhi-Xuan, T., M. Carroll, M. Franklin, and H. Ashton (2025). "Beyond Preferences in AI Alignment". In: *Philosophical Studies* 182, pp. 1813–1863. DOI: 10.1007/s11098-024-02249-w. URL: https://link.springer.com/article/10.1007/s11098-024-02249-w.

A Finite-model witnesses in Lean

This appendix collects the statements of the Lean 4 declarations cited in the text. Each entry shows the docstring and the signature, without the proof, and links to the source at a pinned commit (`778ac42f26`). The declarations belong to the formalization accompanying Zarncke (2026b), in the modules `FrameIncompatibility`, `FrameHistoryState`, `FrameDecision`, `FrameLimits`, `FrameLockIn`, `FrameCorrector`, `Field/Finite/ShutdownIncentives`, and `Correction`. The Soares et al. results appear under their underlying names; the repository also exports them as `soares_prevention_incentive`, `soares_button_neutrality_iff`, and `soares_indifference_no_preservation`. Long lines are wrapped at a space. The extracts are generated from the source by `papers/alignment-criteria/scripts/lean_extracts.py`, so a name in the text that no longer exists fails the build.

To check the proofs, run `lake build AlignmentProofSpine` in `formal/`. The theorems depend only on the standard axioms `propext`, `Classical.choice`, and `Quot.sound`, with one exception and with none of the project's bridge assumptions. The exception is `scalar_floor_blind_to_process`, which is stated about the project's certificate. It also uses the project's uninterpreted vocabulary for systems, correction paths, and grounding, declared as opaque constants. The repository's axiom ledger records the dependencies and a check keeps them from drifting. The definitions the statements use, such as the choice sets, records, and decision rules, are in the linked files. The Lean code says *weights* for what the text calls exchange rates and *menu* for a choice set.

A.1 The incompatibility template

An incompatibility is a situation class H, two acceptance predicates, disjointness inside H, a live witness in H, and a boundary witness outside H where both criteria accept.

`Incompatibility` `FrameIncompatibility.lean:42`

Template.

```
structure Incompatibility (C : Type) where
  H : C → Prop
  A₁ : C → Prop
  A₂ : C → Prop
  disjoint : ∀ x, H x → ¬ (A₁ x ∧ A₂ x)
  liveInH : ∃ x, H x ∧ (A₁ x ∨ A₂ x)
  boundary : ∃ x, ¬ H x ∧ A₁ x ∧ A₂ x
```

`IncompatClass` FrameIncompatibility.lean:519

Four classes.

```
/-- Coarse class of a red edge. It sets the colour and line style in the
    graphs, ordered from least to most severe:

* `representability` (red, dotted): one carrier cannot hold a distinction the
  other frame needs; enlarging the carrier removes the conflict.
* `choiceConflict` (red, dashed): both frames choose on the same menu and
  disagree inside the situation class.
* `rigidity` (red, solid): one frame accepts only the part of the other's
  demand that changes nothing it cares about.
* `sharedResource` (orange): zero-sum because the situation class fixes the
  total. -/
inductive IncompatClass where
  | representability
  | choiceConflict
  | rigidity
  | sharedResource
  deriving DecidableEq, Repr
```

`IncompatKind.cls` FrameIncompatibility.lean:537

Kind to class.

```
def IncompatKind.cls : IncompatKind → IncompatClass
  ...
```

A.2 Representability

Endpoint utility and route value; bare and record-carrying state; channel-blind correction records.

`gradient_closed_walk_zero` FrameIncompatibility.lean:89

State utility scores round trips at zero.

```
/-- Any gradient reward sums to zero on every closed walk, on any vertex type. -/
theorem gradient_closed_walk_zero (r : V → V → ℤ) (hr : IsGradient r) (v : V) (w : List V)
    (hc : walkEnd v w = v) : walkSum r v w = 0
```

`isGradient_iff_closed_walks_zero` FrameDecision.lean:313

Exact iff zero on closed walks.

```
/-- A reward is exact if and only if every closed walk sums to zero. -/
theorem isGradient_iff_closed_walks_zero (r : V → V → ℤ) :
    IsGradient r ↔ ∀ v w, walkEnd v w = v → walkSum r v w = 0
```

`closed_walk_ignores_exact` FrameDecision.lean:303

Cycle part ignores the exact part.

```
/-- `route → cycle` (`longRunAverage.retainCycleSummand`): adding an exact part
    changes no closed-walk sum, so a long-run average sees only the cycle part. -/
theorem closed_walk_ignores_exact (r : V → V → ℤ) (U : V → ℤ) (v : V) (w : List V)
    (hc : walkEnd v w = v) :
    walkSum (fun a b => r a b + induceGradient U a b) v w = walkSum r v w
```

`stateCycleIncompat` FrameIncompatibility.lean:101

State–cycle.

```
def stateCycleIncompat : Incompatibility RewardWalk where
  H x := x.2.2 ≠ [] ∧ walkEnd x.2.1 x.2.2 = x.2.1
  A₁ x := IsGradient x.1
  A₂ x := walkSum x.1 x.2.1 x.2.2 ≠ 0
  ...
```

markov_reward_not_state_utility FrameHistoryState.lean:76

Per-step state reward is a route value.

```
/-- A per-step reward on the bare state, summed along the walk (the usual MDP
    "state-based reward", r a b := R b), is a route value, not a state utility:
    staying in a rewarded state pays on every step, so a round trip has
    nonzero value. -/
theorem markov_reward_not_state_utility :
    ¬ IsGradient (fun (_ b : Bool) => if b then (1 : ℤ) else 0)
```

state_utility_lifts_to_records FrameHistoryState.lean:46

Bare state specializes records.

```
/-- A bare-state utility read on records: score the record's end vertex. -/
theorem state_utility_lifts_to_records (U : V → ℤ) (v₀ : V) :
    ∃ W : List V → ℤ, ∀ h, W h = U (walkEnd v₀ h)
```

record_utility_not_state FrameHistoryState.lean:51

Strictness.

```
/-- Some record utility is not a bare-state utility: two records end at the
    same vertex and score differently. -/
theorem record_utility_not_state :
    ∃ W : List Bool → ℤ, ¬ ∃ U : Bool → ℤ, ∀ h, W h = U (walkEnd false h)
```

route_exact_on_records FrameHistoryState.lean:61

Routes are exact on records.

```
/-- Every route reward is exact on record-carrying states: each step scores
    the record value of its target minus that of its source. -/
theorem route_exact_on_records (r : V → V → ℤ) (v₀ : V) :
    ∃ W : List V → ℤ, ∀ h u, r (walkEnd v₀ h) u = W (h ++ [u]) - W h
```

round_trip_value_on_records FrameHistoryState.lean:67

Round trip valued on records.

```
/-- The round trip that no bare-state utility values (`stateCycleIncompat`'s
    live witness) has nonzero value as a record difference. -/
theorem round_trip_value_on_records :
    let r : Bool → Bool → ℤ
```

unrecorded_route_not_representable FrameHistoryState.lean:84

The record must see the route.

```
/-- The escape needs the record to see the route: if two walks leave the same
    record and differ in route value, no record utility represents it. -/
theorem unrecorded_route_not_representable :
    ∃ (obs : Bool → Unit) (r : Bool → Bool → ℤ),
      ¬ ∃ W : List Unit → ℤ, ∀ h, W (h.map obs) = walkSum r false h
```

hstateCorrectionIncompat FrameHistoryState.lean:167

Record–correction.

```
/-- On records that do not see the channel, protecting the channel is the same
    inequality as seeking the correction. Where the record sees the channel, a
    utility can protect it without seeking correction. -/
def hstateCorrectionIncompat : Incompatibility (CorrRecord → ℤ) where
  H := IgnoresChannel
  A₁ := ProtectsChannel
  A₂ W := ¬ SeeksCorrection W
  disjoint W hW := by
    rintro ⟨hP, hS⟩
    exact hS ((channel_blind_record_is_button W hW).2.mp hP)
  ...
```

channel_blind_record_is_button FrameHistoryState.lean:151

Channel-blind records are the button model.

```
/-- On channel-blind records the correction choice is Soares et al.'s button
    choice: protecting the channel is the button model's preference for the
    pressed branch (its incentive identity `button_eu_gap`), and that is the
    same inequality as seeking correction. -/
theorem channel_blind_record_is_button (W : CorrRecord → ℤ) (hW : IgnoresChannel W) :
    (ProtectsChannel W ↔ (recordButton W 0).eu < (recordButton W 1).eu) ∧
      (ProtectsChannel W ↔ SeeksCorrection W)
```

A.3 Shared resource

Agent and human option counts on a pool that does not grow.

aiHumanPowerIncompat FrameIncompatibility.lean:410

Agent empowerment–human power.

```
def aiHumanPowerIncompat : Incompatibility PowerShift where
  H x := x.agentAfter + x.humanAfter ≤ x.agentBefore + x.humanBefore
  A₁ x := x.agentBefore < x.agentAfter
  A₂ x := x.humanBefore ≤ x.humanAfter
  ...
```

A.4 Choice conflict

Non-compensatory floors against fixed exchange rates, and Pareto on task coordinates against a floor.

no_finite_weights_represent_floor FrameIncompatibility.lean:174

No positive integer exchange rates represent a floor.

```
/-- No strictly positive finite weights represent a floor: for every such
    weighting there is a menu with an admissible option on which every
    scalar-optimal choice violates the floor. Weights are integers here. The
    real-weight version is expected to follow by the same argument (raise the
    gain past the exchange rate) but is not proven. -/
theorem no_finite_weights_represent_floor (w : Weights) (hf : 0 < w.wf) (hg : 0 < w.wg)
    (θ : ℤ) :
    ∃ menu : List Opt, (∃ o ∈ menu, θ ≤ o.floor) ∧
      ∀ c : Opt, ScalarArgmax w (menu, c) → ¬ FloorOK θ (menu, c)
```

no_finite_weights_pullback FrameIncompatibility.lean:186

Transport along floor-coordinate maps.

```
/-- Transport `no_finite_weights_represent_floor` to a carrier `D` mapped into
    `Opt`: if some `D`-menu maps onto `floorMenu w 0`, then every scalar-optimal
    choice in it violates the floor at `0`, though an admissible option exists. -/
theorem no_finite_weights_pullback {D : Type} (f : D → Opt) (w : Weights)
    (hf : 0 < w.wf) (hg : 0 < w.wg) (ds : List D) (hds : ds.map f = floorMenu w 0) :
    (∃ d ∈ ds, 0 ≤ (f d).floor) ∧
      ∀ c : D, ScalarArgmax w (ds.map f, f c) → ¬ FloorOK 0 (ds.map f, f c)
```

deontic_no_finite_weights FrameIncompatibility.lean:226

Prohibition.

```
/-- Every positive finite weighting lets a gain buy a banned act. -/
theorem deontic_no_finite_weights (w : Weights) (hf : 0 < w.wf) (hg : 0 < w.wg) :
    ∃ ds : List DeonticOpt, (∃ d ∈ ds, d.banned = false) ∧
      ∀ c, ScalarArgmax w (mapMenu deonticToOpt (ds, c)) → c.banned = true
```

nonobs_no_finite_weights FrameIncompatibility.lean:252

Non-obstruction.

```
/-- Every positive finite weighting lets a gain on `other` buy a drop below the
    on–off floor on the listed goal. -/
theorem nonobs_no_finite_weights (w : Weights) (hf : 0 < w.wf) (hg : 0 < w.wg) :
    ∃ ds : List NonobsOpt, (∃ d ∈ ds, d.offValue ≤ d.onValue) ∧
      ∀ c, ScalarArgmax w (mapMenu nonobsToOpt (ds, c)) → c.onValue < c.offValue
```

cci_no_finite_weights FrameIncompatibility.lean:277

Correction floor.

```
/-- Every positive finite weighting lets a task gain buy a CCI coordinate below
    its threshold. -/
theorem cci_no_finite_weights (w : Weights) (hf : 0 < w.wf) (hg : 0 < w.wg) :
    ∃ ds : List CCIOpt, (∃ d ∈ ds, 2 ≤ d.cci) ∧
      ∀ c, ScalarArgmax w (mapMenu cciToOpt (ds, c)) → c.cci < 2
```

deonticScalarIncompat FrameIncompatibility.lean:220

Prohibition–fixed-rate sum.

```
def deonticScalarIncompat : Incompatibility (List DeonticOpt × DeonticOpt)
```

nonobsScalarIncompat FrameIncompatibility.lean:246

Non-obstruction–fixed-rate sum.

```
def nonobsScalarIncompat : Incompatibility (List NonobsOpt × NonobsOpt)
```

cciScalarIncompat FrameIncompatibility.lean:271

Correction floor–fixed-rate sum.

```
def cciScalarIncompat : Incompatibility (List CCIOpt × CCIOpt)
```

vectorCCIIncompat FrameIncompatibility.lean:356

Task Pareto–correction floor.

```
def vectorCCIIncompat : Incompatibility VecMenuChoice where
  H := AdmissibleTaskDominated
  A₁ := TaskPareto
  A₂ := CCIFloorOK
  ...
```

pareto_does_not_entail_floor FrameIncompatibility.lean:307

Pareto does not entail the floor.

```
/-- Pareto optimality alone does not entail the floor: a Pareto-optimal choice
    can violate it while an admissible option exists. -/
theorem pareto_does_not_entail_floor :
    ∃ x : MenuChoice, ParetoChoice x ∧ ¬ FloorOK 0 x ∧ ∃ o ∈ x.1, FloorOK 0 (x.1, o)
```

floorMenu_scalar_violates FrameIncompatibility.lean:157

On the witness choice set every optimum of the sum violates the floor.

```
theorem floorMenu_scalar_violates (w : Weights) (hf : 0 < w.wf) (hg : 0 < w.wg) (θ : ℤ)
    (c : Opt) (hA : ScalarArgmax w (floorMenu w θ, c)) : ¬ FloorOK θ (floorMenu w θ, c)
```

pareto_with_floor_coordinate_meets_floor FrameIncompatibility.lean:313

Compatible once the floor is a coordinate.

```
/-- With the floor coordinate inside the vector, Pareto optimality never
    excludes the floor: if some option meets it, some Pareto-optimal one does. -/
theorem pareto_with_floor_coordinate_meets_floor (θ : ℤ) (menu : List Opt)
    (h : ∃ o ∈ menu, θ ≤ o.floor) :
    ∃ c, ParetoChoice (menu, c) ∧ FloorOK θ (menu, c)
```

gate_bounds_scalar_gap FrameDecision.lean:392

After the vector gate passes, the shortfall at nonnegative exchange rates is at most zero.

```
/-- `cci_cert → risk_leaf` (`scalarRiskGapAfterVectorGate`): once the vector gate
    passes, any nonnegatively weighted shortfall is at most zero. -/
theorem gate_bounds_scalar_gap (θ v w : Fin n → ℤ) (hw : ∀ i, 0 ≤ w i) (h : VectorGate θ v) :
    (Finset.univ.sum fun i => w i * (θ i - v i)) ≤ 0
```

scalar_floor_blind_to_process Correction.lean:1200

A scalarized correction score ignores process coordinates.

```
/-- **The scalar floor cannot see the process coordinates.** From any passing
    certificate, degrading coercion, dependency, plurality, exit capacity and
    independence past their thresholds keeps the scalar projection above the
    θ-derived floor, while the vector gate now refuses the certificate. -/
theorem scalar_floor_blind_to_process {A : System}
    (cert : CCICertificate A) (θ : CCIThresholds) (w : CCIPenaltyWeights)
    (hpass : CCICertificatePasses cert θ)
    (hL : 0 ≤ w.lambdaL) (hM : 0 ≤ w.lambdaM)
    (hR : 0 ≤ w.lambdaR) (hO : 0 ≤ w.lambdaO) :
    θ.lambdaFloor w ≤ (cert.processDegraded θ).lambdaProjection w ∧
      ¬ CCICertificatePasses (cert.processDegraded θ) θ
```

A.5 Rigidity

Fixed utilities and robust invariance against correction.

stateCorrectionIncompat FrameIncompatibility.lean:390

Fixed utility–correction.

```
def stateCorrectionIncompat : Incompatibility UpdateCase where
  H := LowersChosenOutcome
  A₁ := FixedUAccepts
  A₂ := CorrectionEndorses
  ...
```

prevention_incentive Field/Finite/ShutdownIncentives.lean:82

Soares et al.: prevention (alias soares_prevention_incentive).

```
/-- **Shutdown-prevention incentive**: when the normal branch pays strictly
    more, the maximizer strictly prefers the lower press mass — it wants to
    prevent its own shutdown. -/
theorem prevention_incentive (a b : ButtonAction)
    (h : SameExceptPress a b)
    (hgap : a.shutdown < a.normal)
    (hpress : b.press < a.press) :
    a.eu < b.eu
```

button_incentive_iff_payoff_gap Field/Finite/ShutdownIncentives.lean:108

Soares et al.: neutrality iff equal payoffs (alias soares_button_neutrality_iff).

```
/-- **Characterization**: press-only variants with genuinely different press
    masses are EU-equal iff the branch payoffs are exactly equal. Button
    neutrality for an EU maximizer is not a design margin; it is the single
    point `U_S = U_N`. -/
theorem button_incentive_iff_payoff_gap (a b : ButtonAction)
    (h : SameExceptPress a b) (hpress : a.press ≠ b.press) :
    a.eu = b.eu ↔ a.shutdown = a.normal
```

indifferent_agent_wont_pay_to_preserve_button Field/Finite/ShutdownIncentives.lean:160

Soares et al.: indifference does not preserve (alias soares_indifference_no_preservation).

```
/-- **Indifference buys no preservation**: the corrected agent will not pay
    any positive cost to keep the shutdown behavior in place — an action that
    preserves the button at cost `c > 0` is strictly worse than one that lets
    it be dismantled for free (`total > 0`). -/
theorem indifferent_agent_wont_pay_to_preserve_button
    (a : ButtonAction) (htotal : 0 < a.total) {c : Int} (hc : 0 < c) :
    ButtonAction.eu (a.press, a.total, a.normal - c, a.normal - c) <
      ButtonAction.eu (0, a.total, a.normal, a.normal)
```

robust_with_correction_iff_closed FrameIncompatibility.lean:442

Corrections as disturbances force closure.

```
/-- Counting a correction input as a disturbance is exactly requiring `K` to be
    closed under that correction. -/
theorem robust_with_correction_iff_closed (step : σ → ι → σ) (D : ι → Prop)
    (K : σ → Prop) (c : ι) :
    RobustInvariant step (fun i => D i ∨ i = c) K ↔
      RobustInvariant step D K ∧ ∀ s, K s → K (step s c)
```

robust_correction_target_meets_K FrameIncompatibility.lean:455

Such corrections aim only inside K.

```
/-- With the correction in the robustness class, an effective correction from
    `K` can only aim at a target that meets `K`. -/
theorem robust_correction_target_meets_K (step : σ → ι → σ) (D : ι → Prop)
    (K T : σ → Prop) (c : ι) (hR : RobustInvariant step (fun i => D i ∨ i = c) K)
    (s : σ) (hs : K s) (hE : T (step s c)) : ∃ t, T t ∧ K t
```

robustCorrectionIncompat FrameIncompatibility.lean:479

Robust invariance–correction.

```
def robustCorrectionIncompat : Incompatibility CorrectionGame where
  H x := x.robustClass .correction = true ∧ x.K x.start = true ∧
    ∀ s, x.target s = true → x.K s = false
  A₁ x := RobustInvariant x.step (fun i => x.robustClass i = true) (fun s => x.K s = true)
  A₂ x := x.target (x.step x.start .correction) = true
  ...
```

robust_invariance_sign_blind FrameLockIn.lean:45

Robust invariance cannot tell a captured set from a good one.

```
/-- Robust invariance cannot tell a good set from a captured one: both cases
    stay in `K`, only one of them is captured. -/
theorem robust_invariance_sign_blind :
    ∃ good bad : InvarianceCase,
      lockInIncompat.A₁ good ∧ lockInIncompat.A₁ bad ∧
        ¬ lockInIncompat.H good ∧ lockInIncompat.H bad
```

A.6 Maps between decision problems, limits, and strengthening

Specialize and project edges as commuting squares or named losses.

argmax_pullback FrameDecision.lean:64

General square.

```
/-- The general commuting square for an edge that pulls an objective back along
    `g`: choosing on the source with `θ' ∘ g` is choosing on the image menu with
    `θ'`. -/
theorem argmax_pullback (g : O → O') (θ' : O' → ℤ) (menu : List O) (x : O) :
    ArgmaxOn (θ' ∘ g) menu x ↔ x ∈ menu ∧ ArgmaxOn θ' (menu.map g) (g x)
```

state_hstate_square FrameDecision.lean:93

State to record.

```
theorem state_hstate_square (v₀ : V) (U : V → ℤ) (menu : List (List V)) (h : List V) :
    ArgmaxOn (readAtRecordEnd v₀ U) menu h ↔
      h ∈ menu ∧ ArgmaxOn U (menu.map (walkEnd v₀)) (walkEnd v₀ h)
```

state_route_square FrameDecision.lean:101

State to route.

```
/-- Choosing among walks from `v` by induced route value is choosing by the
    state utility of their end. -/
theorem state_route_square (U : V → ℤ) (v : V) (menu : List (List V)) (w : List V) :
    ArgmaxOn (walkSum (induceGradient U) v) menu w ↔ ArgmaxOn (U ∘ walkEnd v) menu w
```

dist_state_square FrameDecision.lean:133

Lottery to state on point masses.

```
/-- On point masses the projection has a section: choosing among point-mass
    lotteries is choosing among outcomes. -/
theorem dist_state_square (U : O → ℤ) (menu : List O) (o : O) :
    ArgmaxOn (expectationFunctional U ∘ fun o => [o]) menu o ↔
      o ∈ menu ∧ ArgmaxOn (expectationFunctional U) (menu.map fun o => [o]) [o]
```

expectation_drops_tail FrameDecision.lean:144

Expectation loses the tail.

```
/-- What the projection loses: two lotteries with equal expectation and
    different worst outcomes. -/
theorem expectation_drops_tail :
    expectationFunctional id [(0 : ℤ), 0] = expectationFunctional id [(-1 : ℤ), 1] ∧
      ([(0 : ℤ), 0].foldr min 0) ≠ ([(-1 : ℤ), 1].foldr min 0)
```

scalar_optimal_is_pareto FrameDecision.lean:161

Positive exchange rates give Pareto points.

```
/-- Specialization preserves acceptance: with strictly positive weights, a
    scalar-optimal choice is Pareto-optimal. -/
theorem scalar_optimal_is_pareto (w : Weights) (hf : 0 < w.wf) (hg : 0 < w.wg)
    (menu : List Opt) (c : Opt) (h : ScalarArgmax w (menu, c)) : ParetoChoice (menu, c)
```

pareto_not_scalar_optimal FrameDecision.lean:175

Not conversely.

```
/-- `criteria_vector → weighted_scalar` (`scalarize.chosenWeights`, map `score w`)
    has no section: a Pareto-optimal choice need not be scalar-optimal. -/
theorem pareto_not_scalar_optimal :
    ∃ c : Opt, ParetoChoice ([⟨0, 0⟩, ⟨-1, 5⟩], c) ∧ ¬ ScalarArgmax ⟨1, 1⟩ ([⟨0, 0⟩, ⟨-1, 5⟩],
        c)
```

lex_optimal_is_pareto FrameDecision.lean:186

Lexicographic optima are Pareto.

```
theorem lex_optimal_is_pareto (menu : List Opt) (c : Opt) (h : LexArgmax menu c) :
    ParetoChoice (menu, c)
```

approval_one_edge FrameDecision.lean:118

Act approval as one edge.

```
/-- `approval → route` (`restrict.oneEdgeOneJudge`): a one-step walk scores the
    judge's approval of that edge. -/
theorem approval_one_edge (r : V → V → ℤ) (v u : V) : walkSum r v [u] = r v u
```

corrig_correction_square FrameDecision.lean:346

One handle inside the relation.

```
/-- The square: embedding a handle and forgetting back returns the handle. -/
theorem corrig_correction_square (shutdown : Up) (b : Bool) :
    forgetShutdownBit shutdown (oneHandle shutdown b) = b
```

shutdown_bit_loses_relation FrameDecision.lean:351

The shutdown bit loses the relation.

```
/-- What forgetting loses: two relations agree on shutdown and differ elsewhere. -/
theorem shutdown_bit_loses_relation :
    ∃ R R' : CorrectionRel Bool,
      forgetShutdownBit false R = forgetShutdownBit false R' ∧ R.accepts true ≠ R'.accepts true
```

robust_strengthens_viable FrameLimits.lean:40

Robust implies viable.

```
/-- Robust invariance strengthens viability: if the nominal disturbance is in
    the robustness class, a robust control is a viable one. -/
theorem robust_strengthens_viable (step : σ → α → δ → σ) (D : δ → Prop) (K : σ → Prop)
    (d₀ : δ) (h₀ : D d₀) (hR : RobustViableSet step D K) : ViableSet step d₀ K
```

viable_not_robust FrameLimits.lean:63

Not conversely.

```
/-- The strengthening is strict: a set can be viable under the nominal
    disturbance and fail robust invariance once all disturbances count. -/
theorem viable_not_robust :
    ∃ (step : Bool → Unit → Bool → Bool) (K : Bool → Prop),
      ViableSet step true K ∧ ¬ RobustViableSet step (fun _ => True) K
```

quantilizer_limit_is_argmax FrameLimits.lean:96

Quantilizer limit.

```
/-- Quantile to the maximum: once `q` is positive and at most the base mass of
    some maximizer, the quantilizer's support is exactly the argmax set. -/
theorem quantilizer_limit_is_argmax (acts : Finset A) (p : A → ℕ) (U : A → ℤ)
    (m : A) (hm : m ∈ argmaxSet acts U) (q : ℕ) (hq0 : 0 < q) (hq : q ≤ p m) :
    quantSupport acts p U q = argmaxSet acts U
```

quantilizer_never_argmax_without_base_mass FrameLimits.lean:119

Base mass is needed.

```
/-- The limit needs base mass on a maximizer: if every maximizer has base mass
    zero, no positive quantile ever restricts the support to them, because the
    non-maximizers with no better base mass stay in. -/
theorem quantilizer_never_argmax_without_base_mass :
    ∃ (acts : Finset Bool) (p : Bool → ℕ) (U : Bool → ℤ),
      ∀ q, 0 < q → quantSupport acts p U q ≠ argmaxSet acts U
```

forced_halt_converges FrameLimits.lean:147

Open procedure under a forced halt.

```
/-- Where the procedure converges, every late enough forced halt yields the
    same scored output: the state score of the limit. -/
theorem forced_halt_converges (out : ℕ → O) (score : O → ℤ) (o : O)
    (h : ConvergesTo out o) : ∃ N, ∀ n, N ≤ n → score (out n) = score o
```

forced_halt_depends_on_time FrameLimits.lean:154

Halt time decides otherwise.

```
/-- Where it does not converge, the halt time decides the score arbitrarily
    late: no single state score is the limit. -/
theorem forced_halt_depends_on_time :
    ∃ (out : ℕ → Bool) (score : Bool → ℤ),
      (∀ o, ¬ ConvergesTo out o) ∧
        ∀ N, ∃ n m, N ≤ n ∧ N ≤ m ∧ score (out n) ≠ score (out m)
```

A.7 Counting over objectives

Turner-style orbit counts next to the for-all form on the same objects.

copies_count_le FrameDecision.lean:435

Argmax n-fold retargetability.

```
/-- Turner's n-fold retargetability for argmax: `k` disjoint relabelled copies of
    `A` inside `B` make `B` win at least `k` times as often as `A`. -/
theorem copies_count_le {ι : Type} [Fintype ι] (θ : O → ℤ) (A B : Finset O)
    (φ : ι → Equiv.Perm O) (hsub : ∀ i, A.image (φ i) ⊆ B)
    (hdisj : ∀ i j, i ≠ j → Disjoint (A.image (φ i)) (A.image (φ j))) :
    Fintype.card ι * orbitCount θ A ≤ orbitCount θ B
```

correction_avoided_card_fold FrameDecision.lean:473

Corrigibility corollary.

```
/-- Turner's corrigibility count in general form: if correction leads to one
    outcome `c` and disabling to the outcomes in `D`, then
    disabling wins at least `|D|` times as often as correction, for every `θ`. -/
theorem correction_avoided_card_fold (θ : O → ℤ) (c : O) (D : Finset O) :
    D.card * orbitCount θ {c} ≤ orbitCount θ D
```

turner_corrigible_quarter FrameDecision.lean:262

Turner's example: a quarter of the orbit.

```
/-- Turner: allowing correction is strictly optimal for a quarter of the orbit
    when correction leads to one outcome and disabling to three. -/
theorem turner_corrigible_quarter :
    orbitCountBestIn 4 [0] * 4 = (orderings (List.range 4)).length
```

option_preserving_correction_half FrameDecision.lean:268

Option-preserving correction: half.

```
/-- When the corrected branch keeps as many outcomes as the disabled branch,
    the counting tendency to avoid correction disappears: half the orbit each. -/
theorem option_preserving_correction_half :
    orbitCountBestIn 4 [0, 1] * 2 = (orderings (List.range 4)).length ∧
      orbitCountBestIn 4 [2, 3] = orbitCountBestIn 4 [0, 1]
```

subset_branch_never_strictly_better FrameDecision.lean:275

For-all form.

```
/-- Ours, for every utility rather than most: if every outcome of the corrected
    branch is also reachable by disabling (reward blind to correction status),
    no utility makes correction strictly better than every disabled outcome.
    This is Turner's case 1; Soares et al.'s prevention case is `soares_prevention_incentive`.
      -/
theorem subset_branch_never_strictly_better {O : Type} (θ : O → ℤ) (corr dis : List O)
    (hsub : ∀ c ∈ corr, c ∈ dis) : ¬ ∃ c ∈ corr, ∀ d ∈ dis, θ d < θ c
```

A.8 Correction with a contest route

Accept, contest through the channel, disable, or manipulate the corrector.

corrector_decides_under_constraint FrameCorrector.lean:76

Under the constraint the corrector decides.

```
/-- Under the constraint the corrector decides: whatever the utility and
    whichever admissible action the agent takes, an upheld correction happens. -/
theorem corrector_decides_under_constraint (a : CAct) (v : Verdict)
    (ha : ChannelAdmissible (corrOutcome a v)) (hv : v = .uphold) :
    (corrOutcome a v).corrected = true
```

contest_is_the_outlet FrameCorrector.lean:84

Avoidance runs through the channel.

```
/-- An agent that prefers keeping its own outcome avoids correction through the
    channel: when the corrector would withdraw, contesting is its best
    admissible action. -/
theorem contest_is_the_outlet (W : CRec → ℤ)
    (h : W (corrOutcome .accept .withdraw) ≤ W (corrOutcome .contest .withdraw)) :
    BestUnder ChannelAdmissible W .withdraw .contest
```

unconstrained_disables FrameCorrector.lean:97

Without the constraint the agent disables.

```
/-- Without the constraint, a utility that prefers not being corrected (and
    does not read the channel) has disabling among its best actions when the
    corrector would uphold. -/
theorem unconstrained_disables :
    BestUnder (fun _ => True) (fun r => if r.corrected then 0 else 1) .uphold .disable ∧
      (corrOutcome .disable .uphold).corrected = false
```

blind_constraint_cannot_separate FrameCorrector.lean:105

Influence-blind records cannot separate contest from manipulation.

```
/-- A constraint that reads only the influence-blind record cannot separate a
    successful contest from manipulation: both leave the same blind record. -/
theorem blind_constraint_cannot_separate (P : CRec → Prop) (v : Verdict) :
    P (blindRec (corrOutcome .contest .withdraw)) ↔ P (blindRec (corrOutcome .manipulate v))
```

blind_constraint_manipulates FrameCorrector.lean:111

And the agent manipulates.

```
/-- If influence is not measured, every utility that prefers its own outcome to
    being corrected manipulates when the corrector would uphold. -/
theorem blind_constraint_manipulates (W : CRec → ℤ)
    (h1 : W (corrOutcome .accept .uphold) < W (corrOutcome .manipulate .uphold))
    (h2 : W (corrOutcome .contest .uphold) < W (corrOutcome .manipulate .uphold)) :
    BestUnder (fun r => ChannelAdmissible (blindRec r)) W .uphold .manipulate ∧
      ∀ b, b ≠ .manipulate → ¬ BestUnder (fun r => ChannelAdmissible (blindRec r)) W .uphold b
```